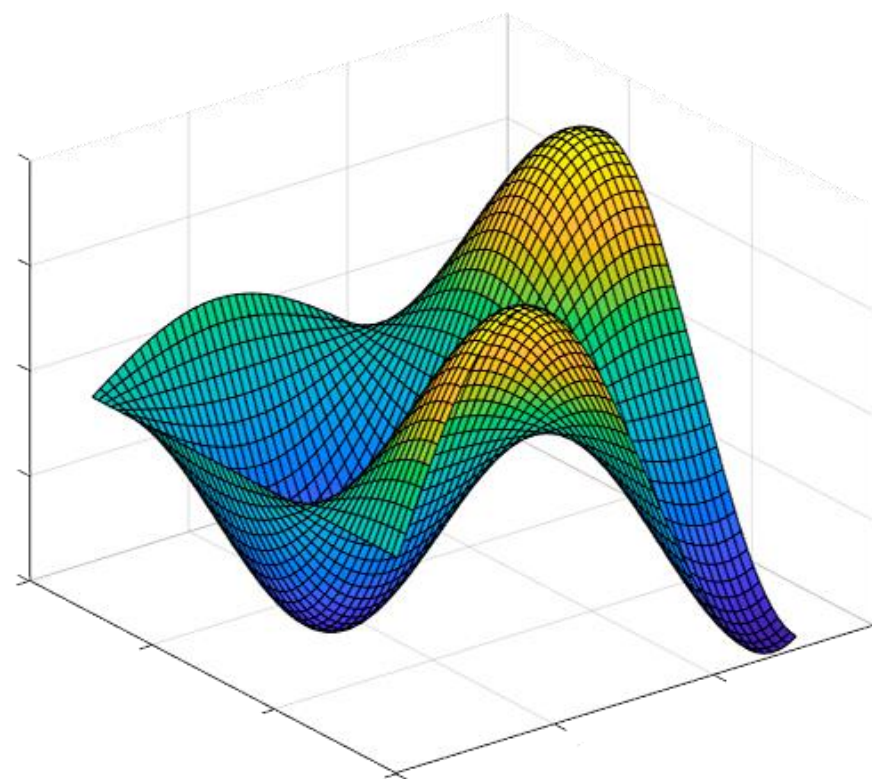


Gradient-Based Optimization

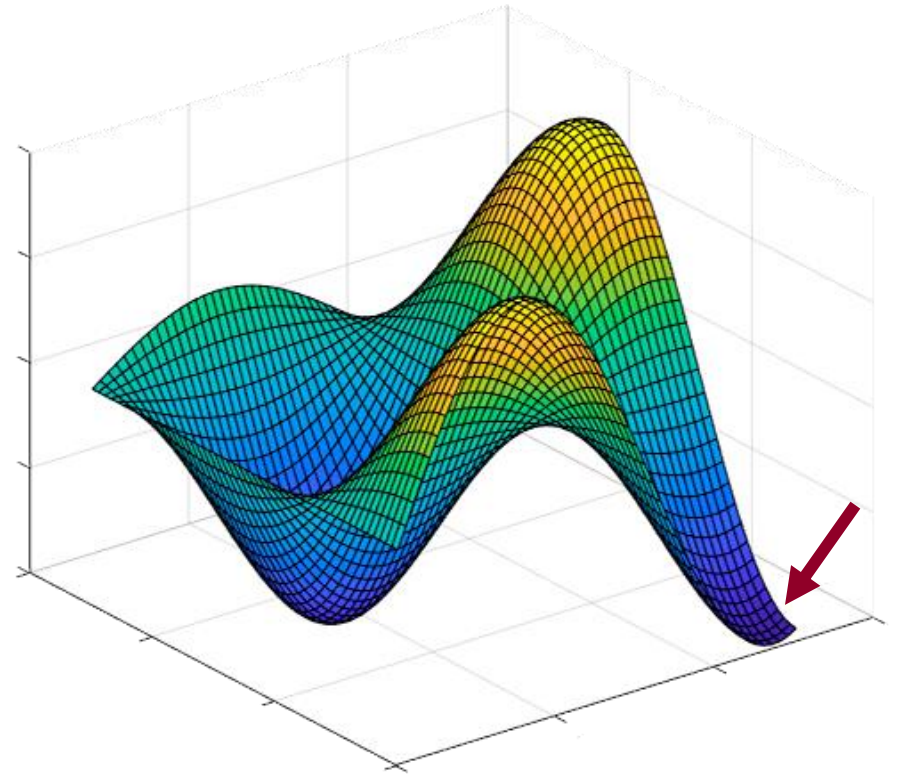
Error as Geometry

- Deep learning algorithms involve optimization of some sort.
- In this context, optimization means **minimizing** some **error function**.
- We can think of the errors in our systems in terms of geometry.



Objective Function

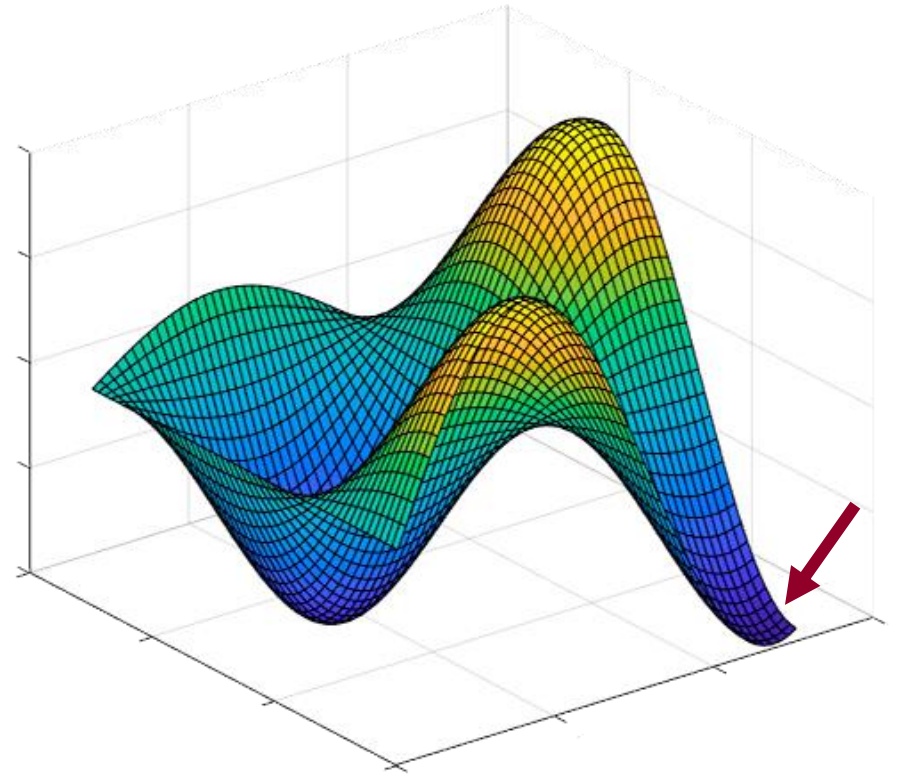
- The function $f(\mathbf{x})$ we want to minimize is called **objective function** (also criterion, cost function, or loss function).
- It is minimized at:
$$\mathbf{x}^* = \arg \min f(\mathbf{x})$$



Function Slope at x

- The function derivative $f'(x)$ gives the slope of $f(x)$ at the point x .
- It tells us how to scale a small change in the input to obtain a corresponding change in the output.

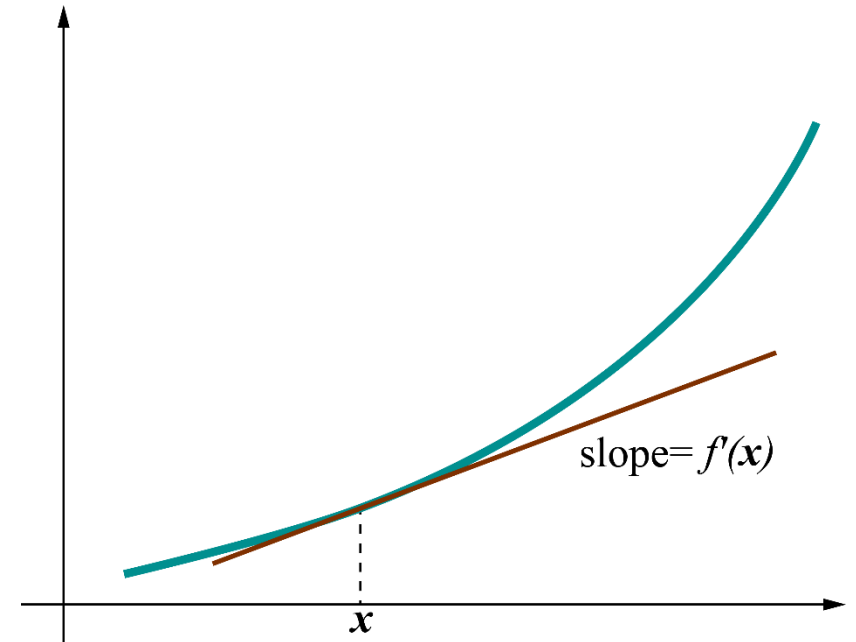
$$f(x + \epsilon) \approx f(x) + \epsilon f'(x)$$



Function Slope at x

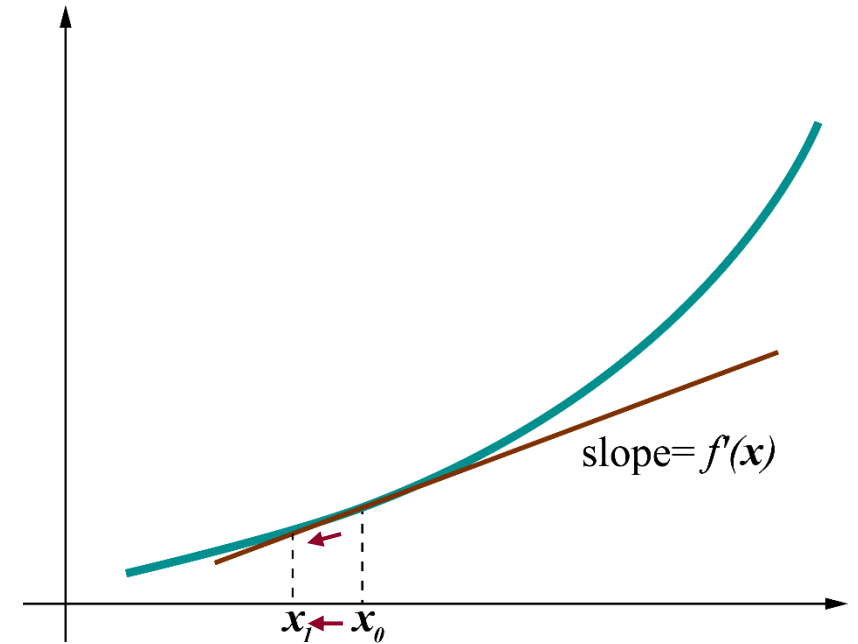
- The function derivative $f'(x)$ gives the slope of $f(x)$ at the point x .
- It tells us how to scale a small change in the input to obtain a corresponding change in the output.

$$f(x + \epsilon) \approx f(x) + \epsilon f'(x)$$



Minimization Strategy

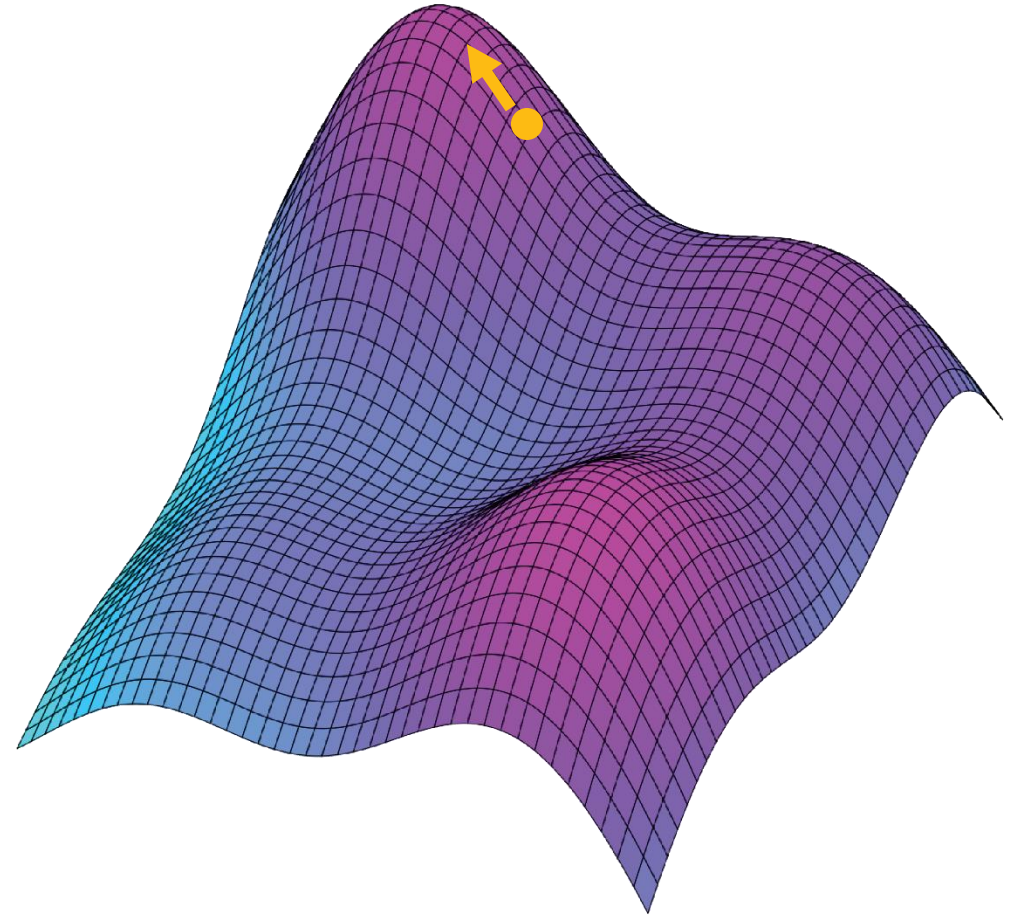
- Minimizing strategy is useful for **minimizing** a function.
- We can find the x minimizing $f(x)$ by moving x in small steps with the opposite sign of the derivative.
- We use the term **gradient** (instead of slope) for functions with several inputs and a single output.



Gradient Vector

- In functions with **multiple inputs**, we need a generalization of the notion of derivative.
- It is a vector with **partial derivatives**.

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{bmatrix}$$



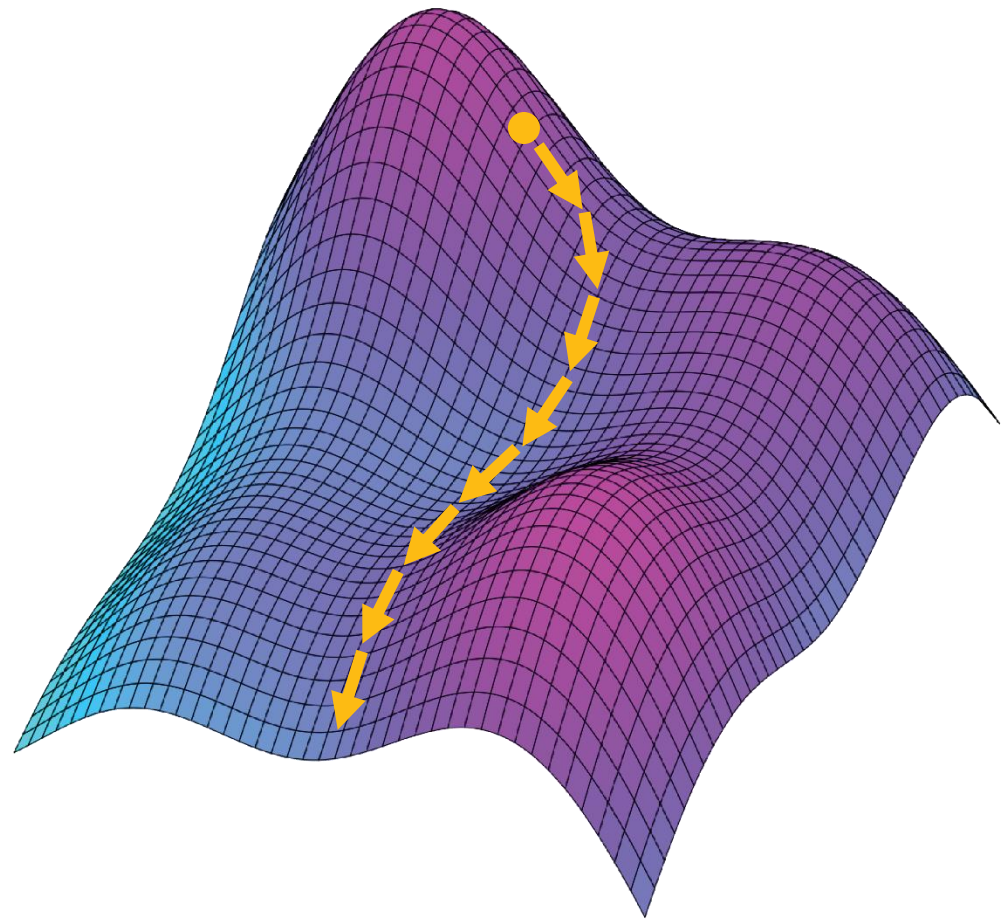
Gradient Descent

- Technique is called **gradient descent** (Cauchy, 1847)
- Update point $\mathbf{x}$ using the negative gradient direction (downhill)

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \epsilon \nabla f(\mathbf{x}_t)$$

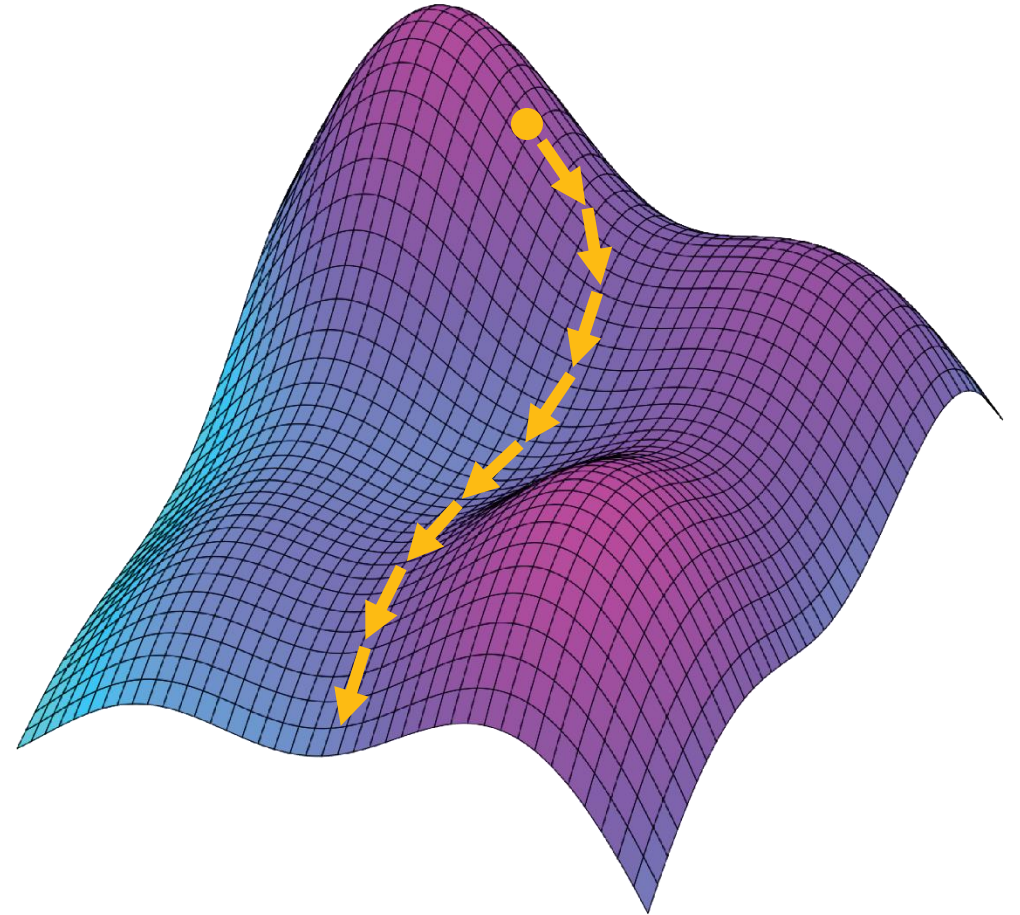
- Monotonic sequence

$$f(\mathbf{x}_t) \geq f(\mathbf{x}_{t+1}) \geq f(\mathbf{x}_{t+2}) \geq \dots$$



Gradient Descent (cont.)

- Technique is called **gradient descent** (Cauchy, 1847)
- Update point $\mathbf{x}$ using the negative gradient direction (downhill)
$$\mathbf{x}_{t+1} = \mathbf{x}_t - \epsilon \nabla f(\mathbf{x}_t)$$
- Converges when gradient is ~ 0



Gradient-Based Optimization

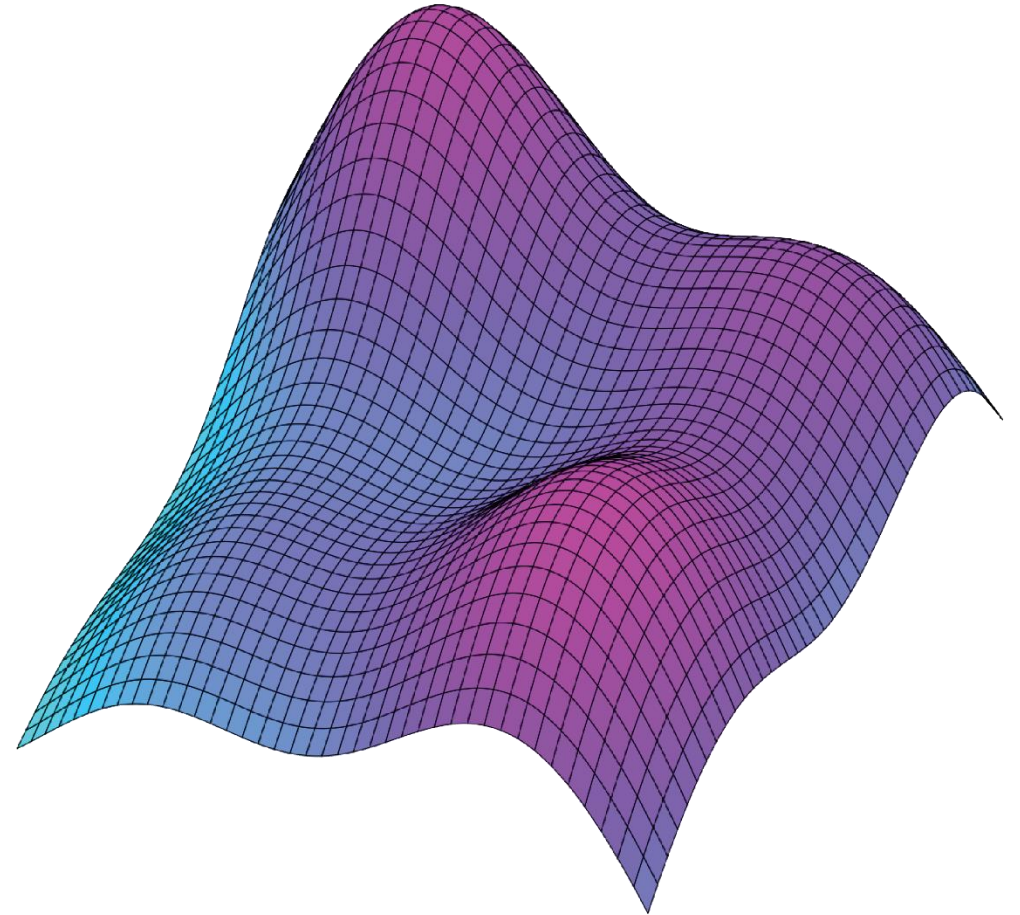
The End

Error Landscape

Shape Features

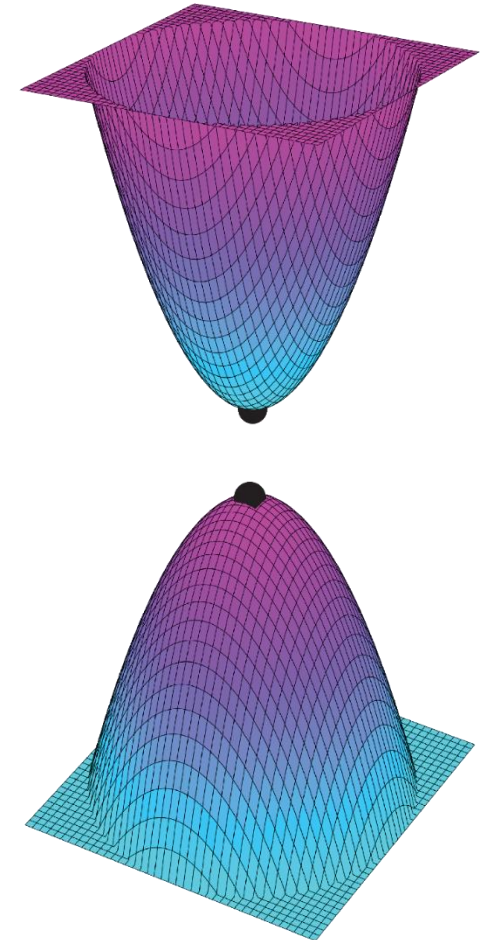
Shape features common to all error surfaces

- Minimum
- Maximum
- Plateau
- Saddle



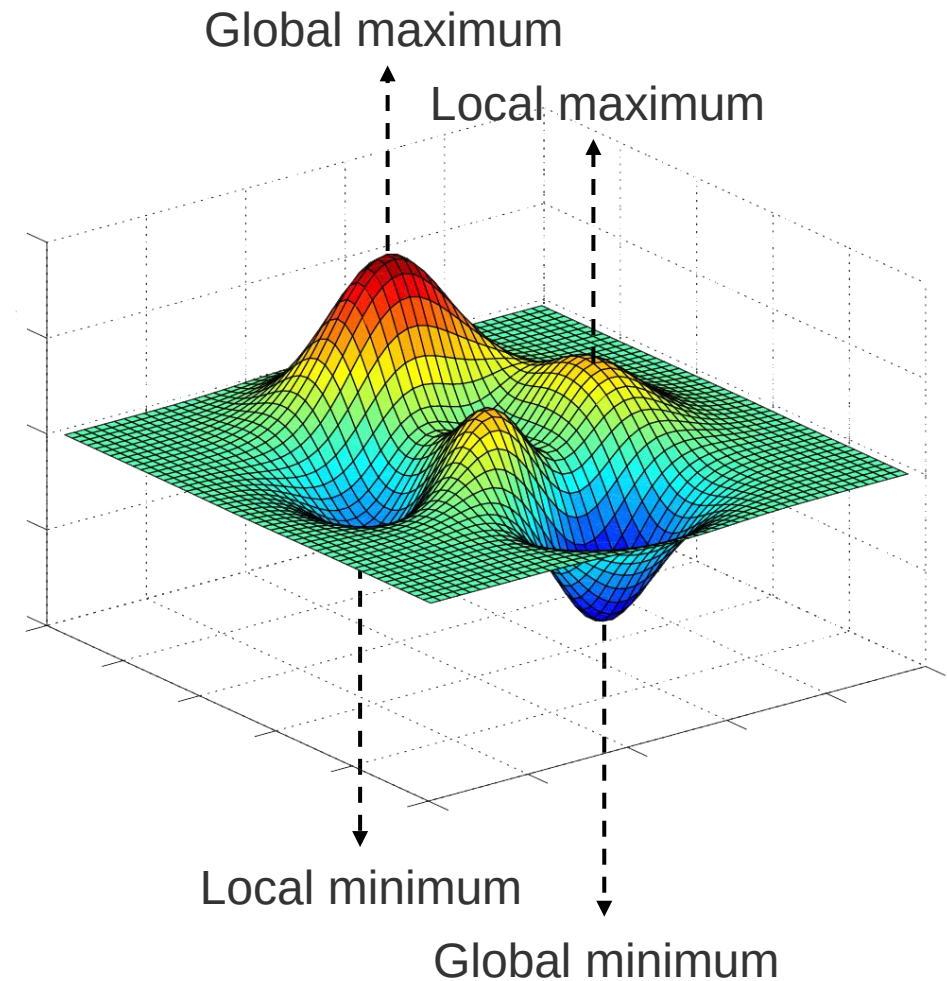
Minimum and Maximum

- A **minimum** is a point on the surface where the gradient is zero and any movement in any direction results in an upward change.
- A hilltop is called the **maximum**; it's a peak where motion in any direction leads downhill.



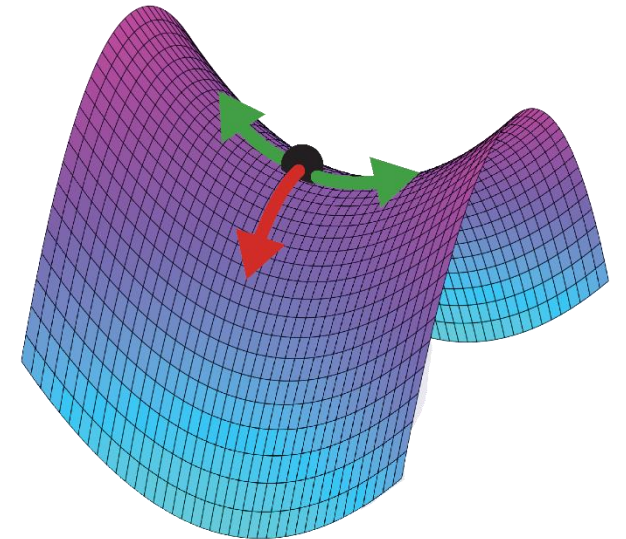
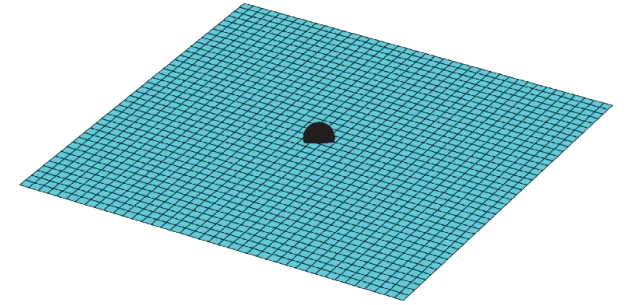
Global vs. Local

- Error functions can have multiple local minima and maxima.
- A point that obtains the absolute lowest value of $f(x)$ is a **global minimum**.
- A point that obtains the absolute highest value of $f(x)$ is a **global maximum**.

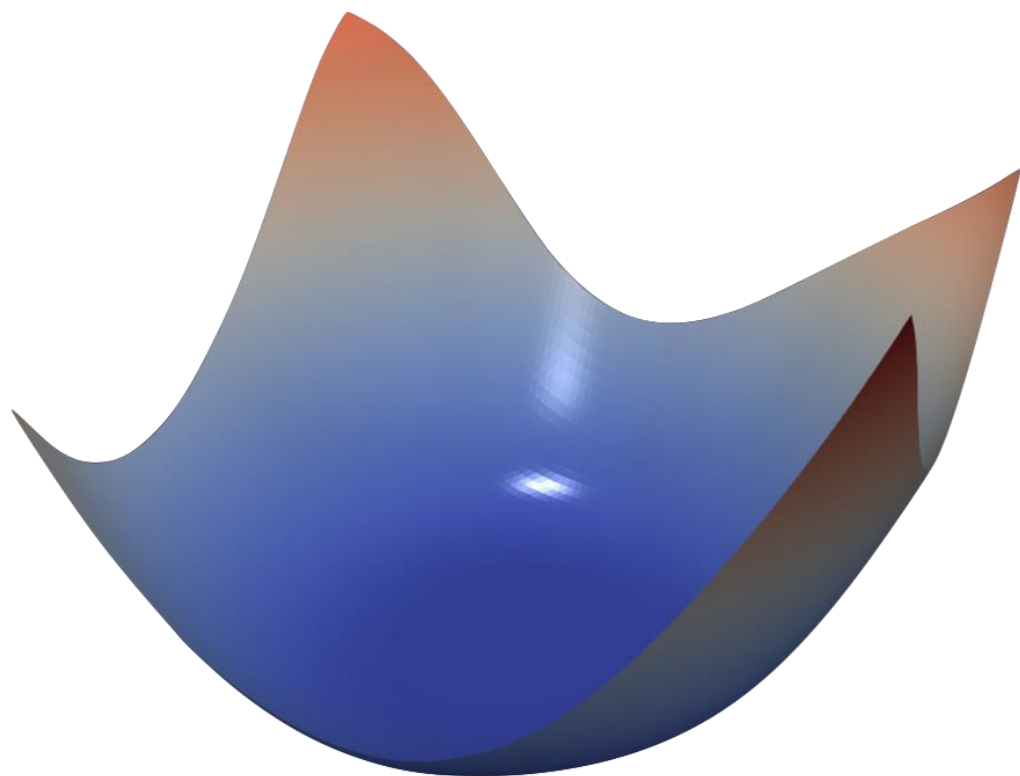


Plateau and Saddle

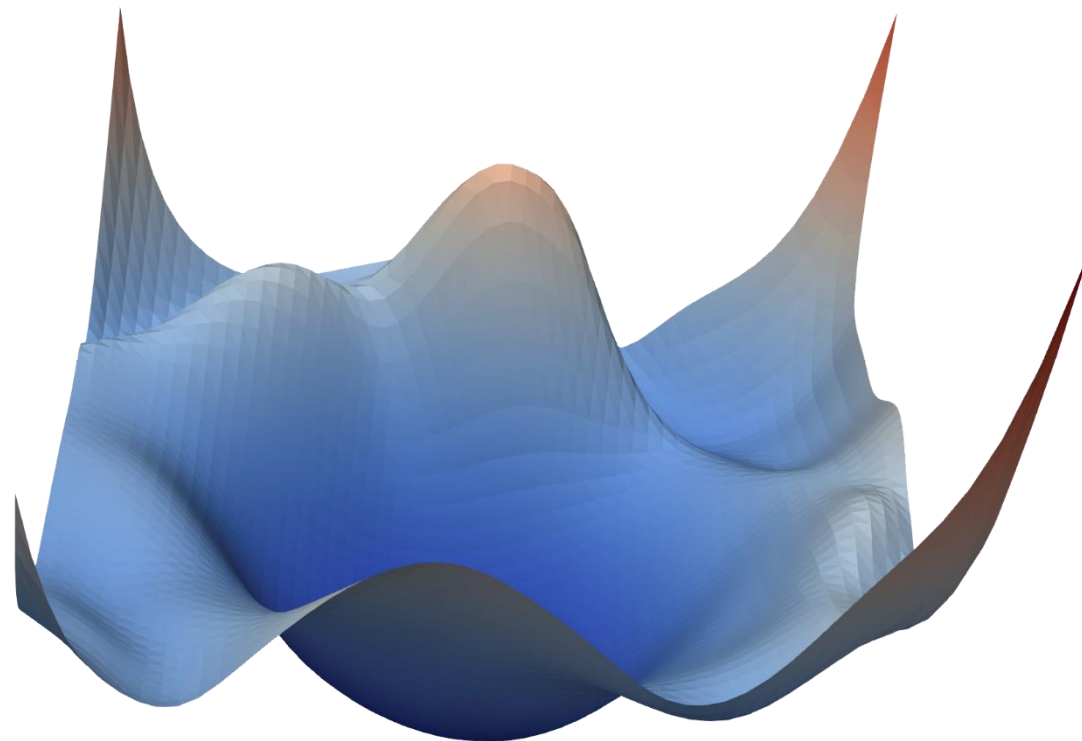
- A **plateau** is an area where the surface is flat, causing no changes in height, regardless of any movement. The gradient is zero in such flat surfaces.
- In a **saddle**, moving along one or more axes results in an increase in error, while moving along one or more other axes leads to a decrease in error.



Loss Landscape

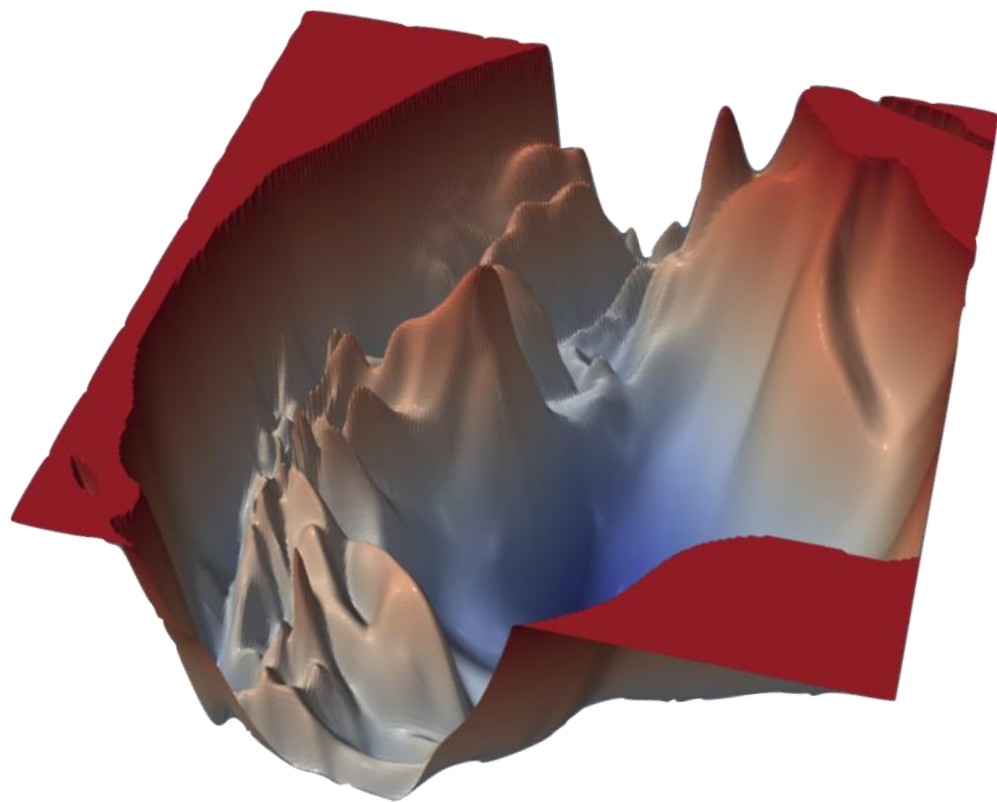


Densenet-121

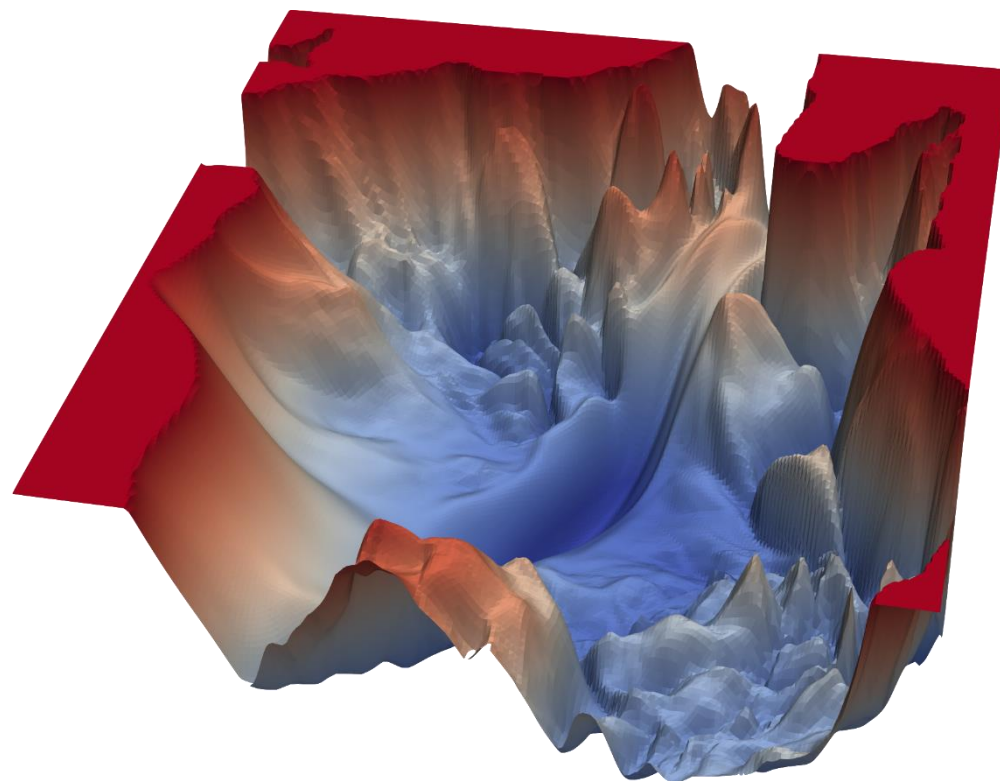


Resnet-56

Loss Landscape (cont.)



VGG-110



VGG-56

Error Landscape

The End

Learning Rate and Schedules

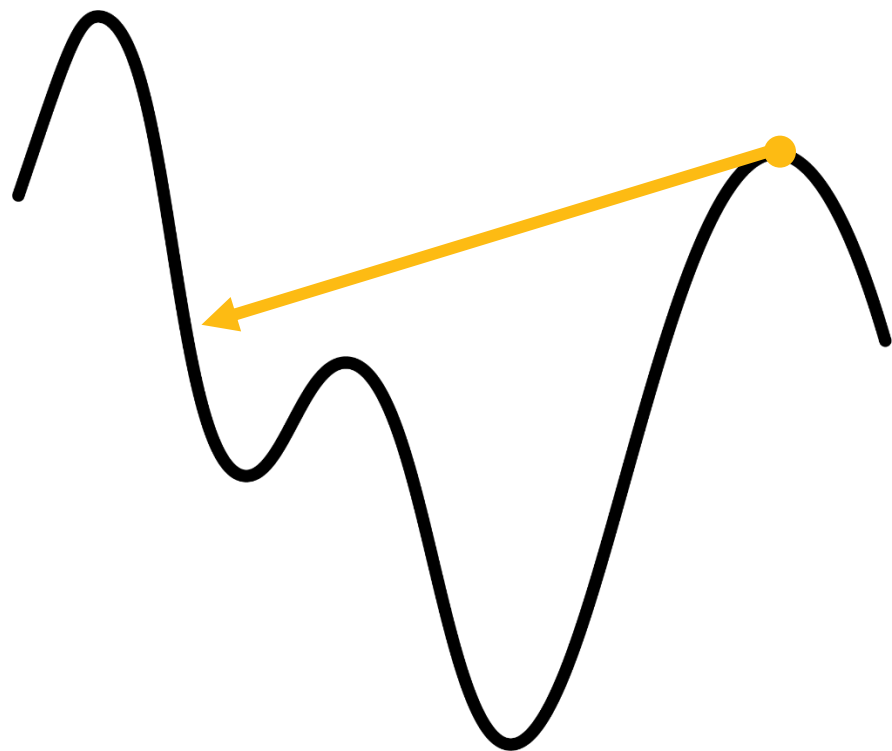
Learning Rate

- Gradient descent's update rule

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \epsilon \nabla f(\mathbf{x}_t)$$

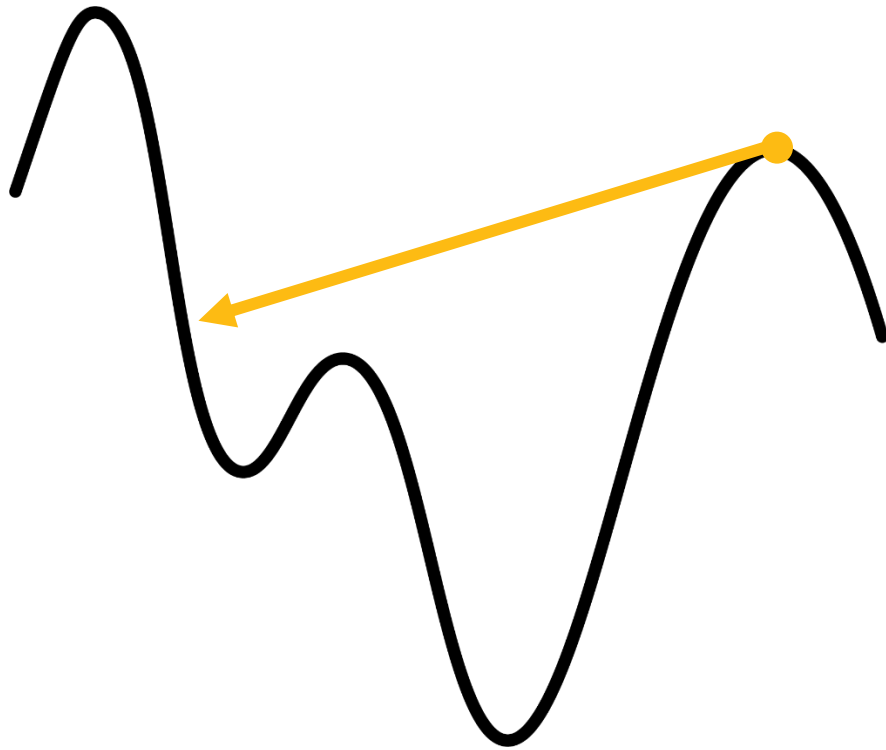
- **Learning rate** (ϵ) is a critical *hyperparameter* when learning with gradient descent
- Determines how much the computed gradients are used to update the weights

Learning Rate (cont.)

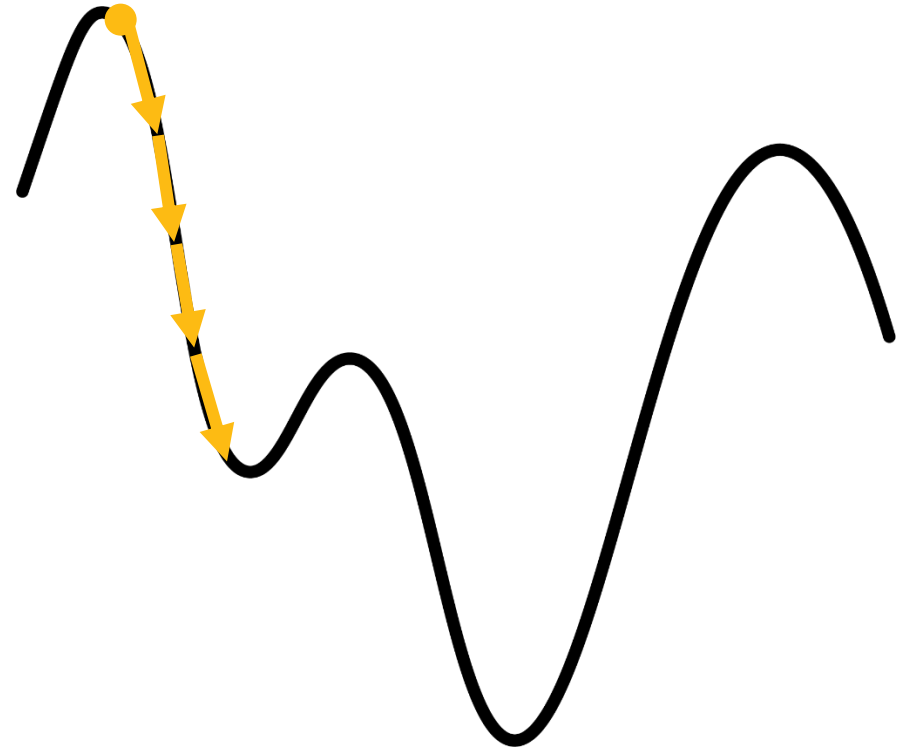


Too large

Learning Rate (cont.)



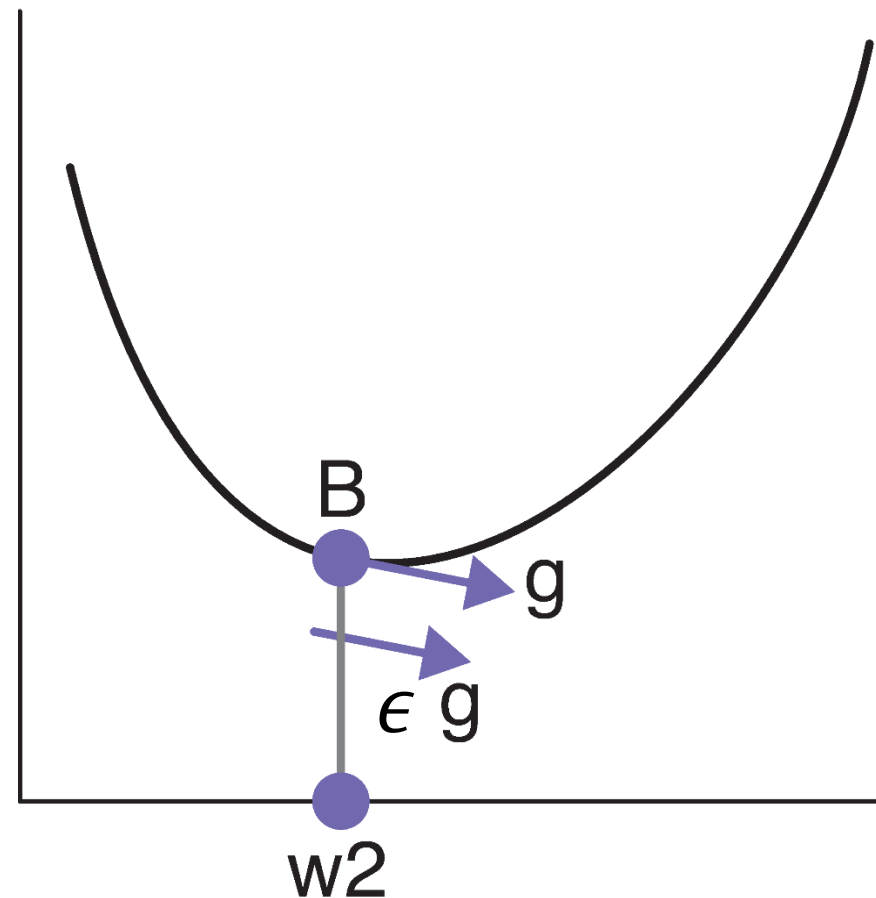
Too large



Too small

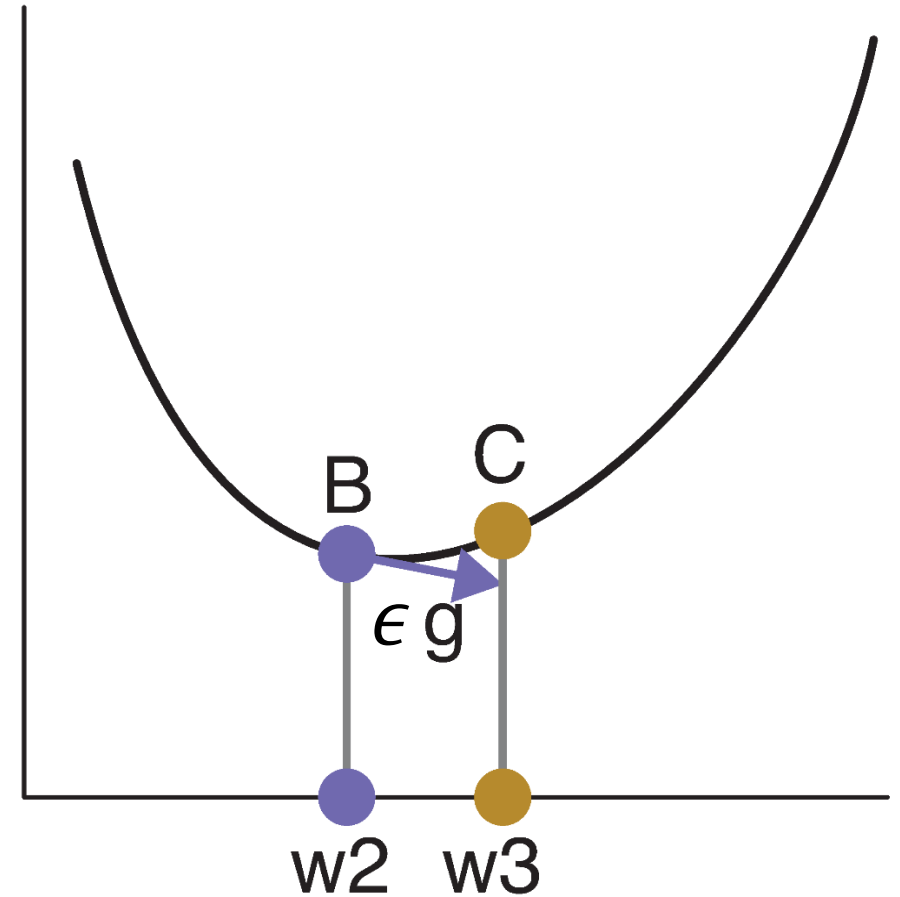
Parameter Space Trajectory

- The trajectory we take is entirely confined to the network parameter space (the weights).

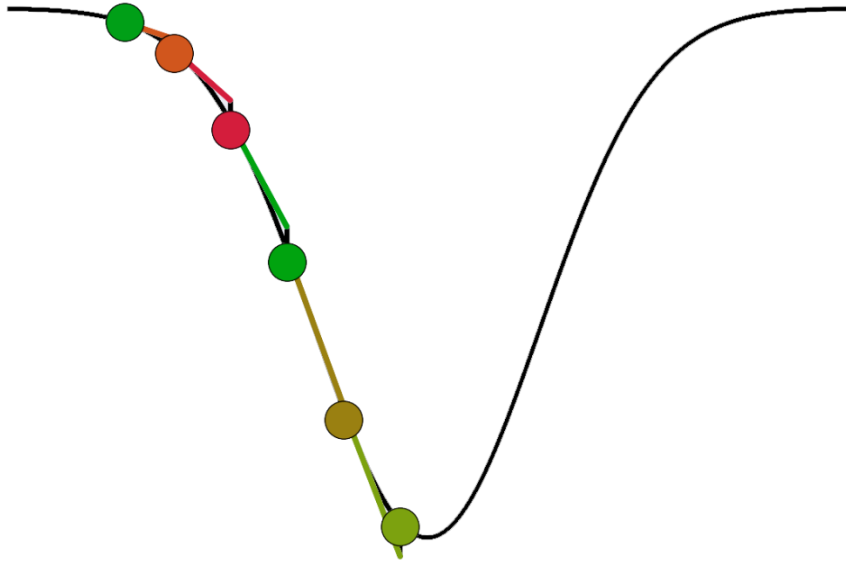


Parameter Space Trajectory (cont.)

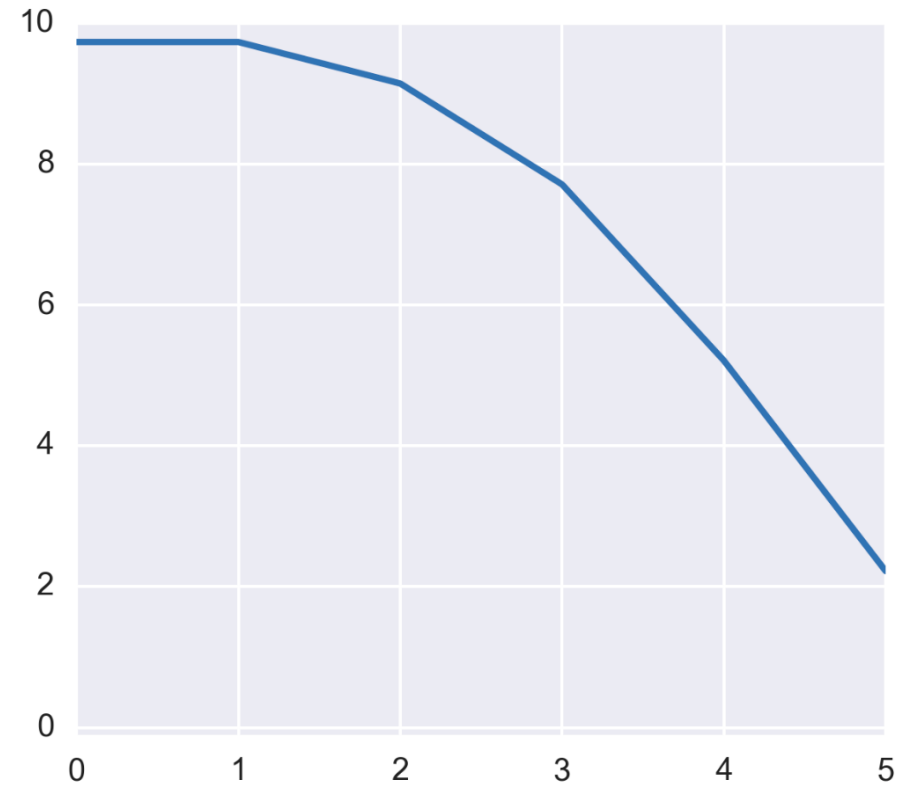
- The trajectory we take is entirely confined to the network parameter space (the weights).
- To find w_3 , the new value of the weight, we add the scaled gradient to w_2 .



Constant Learning Rate, Part I

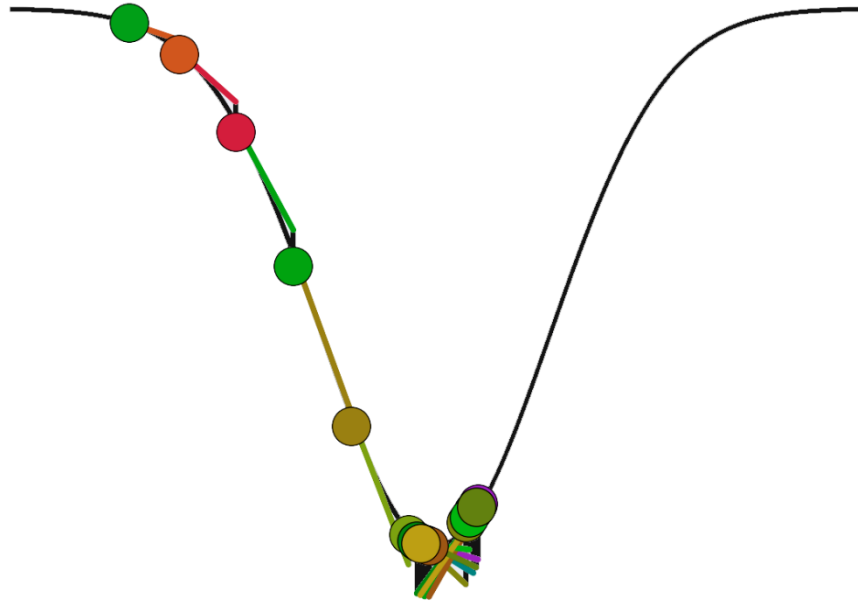


Learning rate $\epsilon = \frac{1}{8}$, six steps

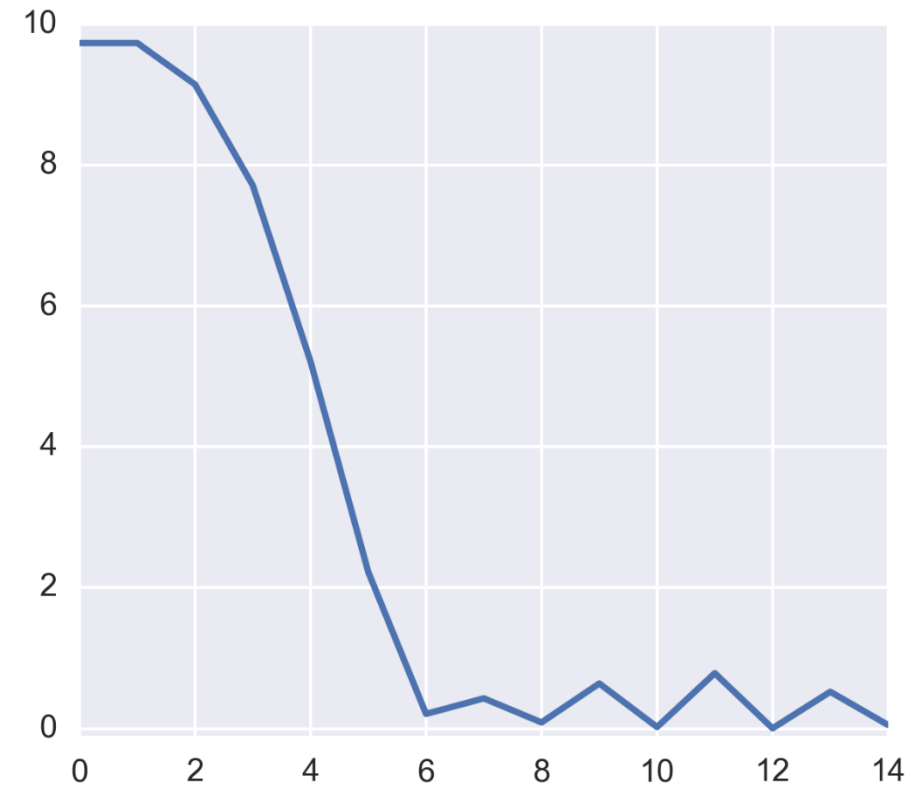


Error curve

Constant Learning Rate, Part II

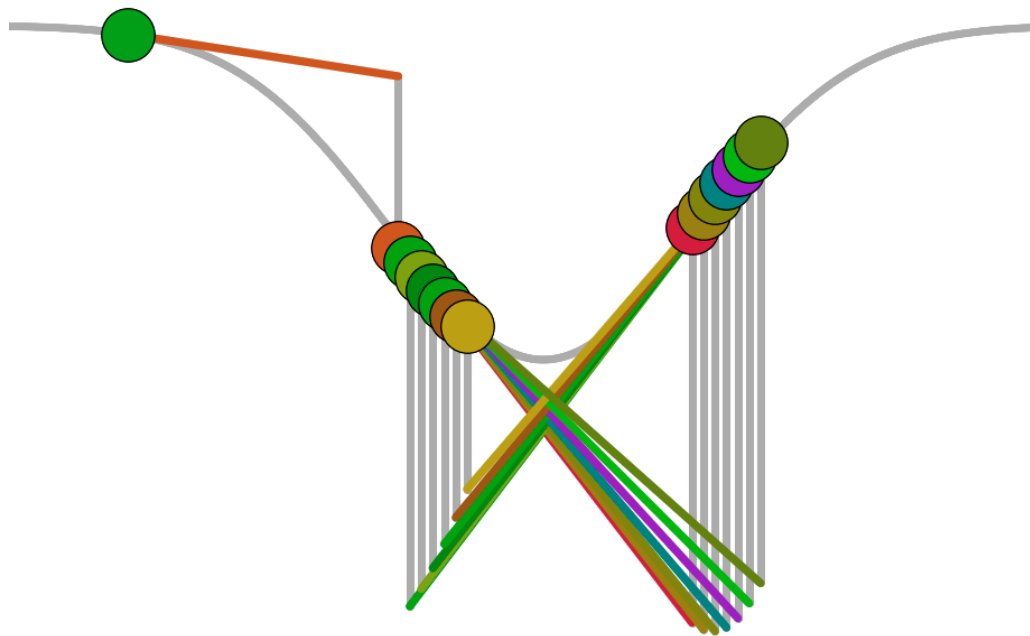


Learning rate $\epsilon = \frac{1}{8}$, 15 steps

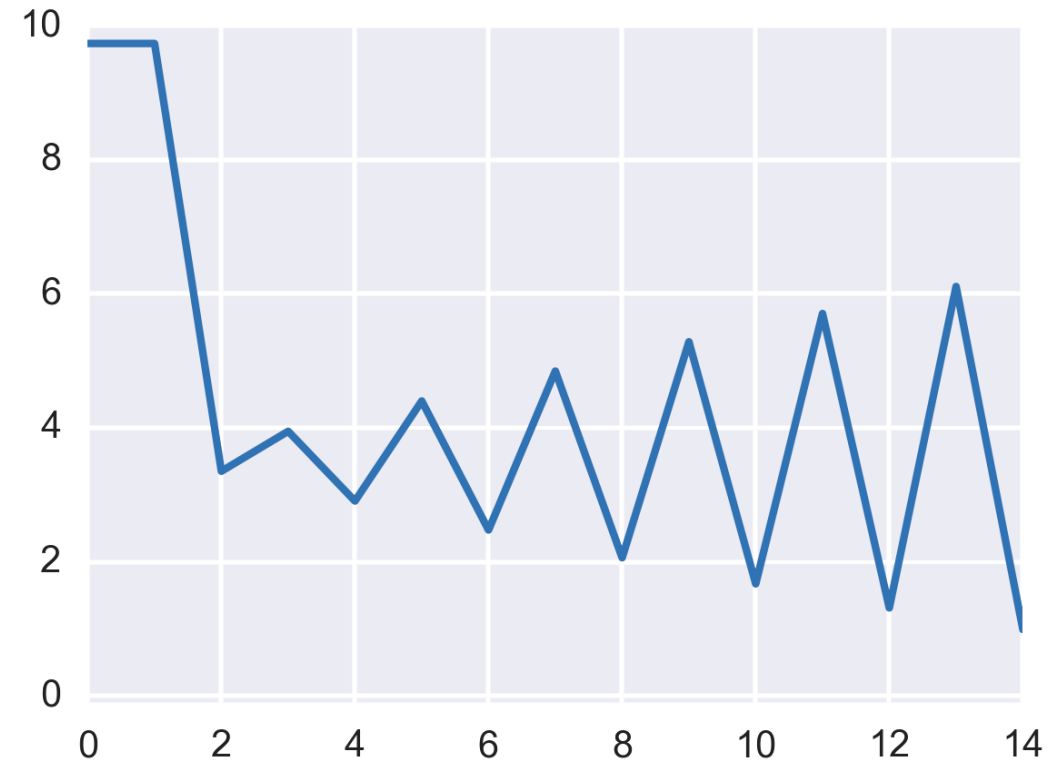


Error curve

Constant Learning Rate, Part III

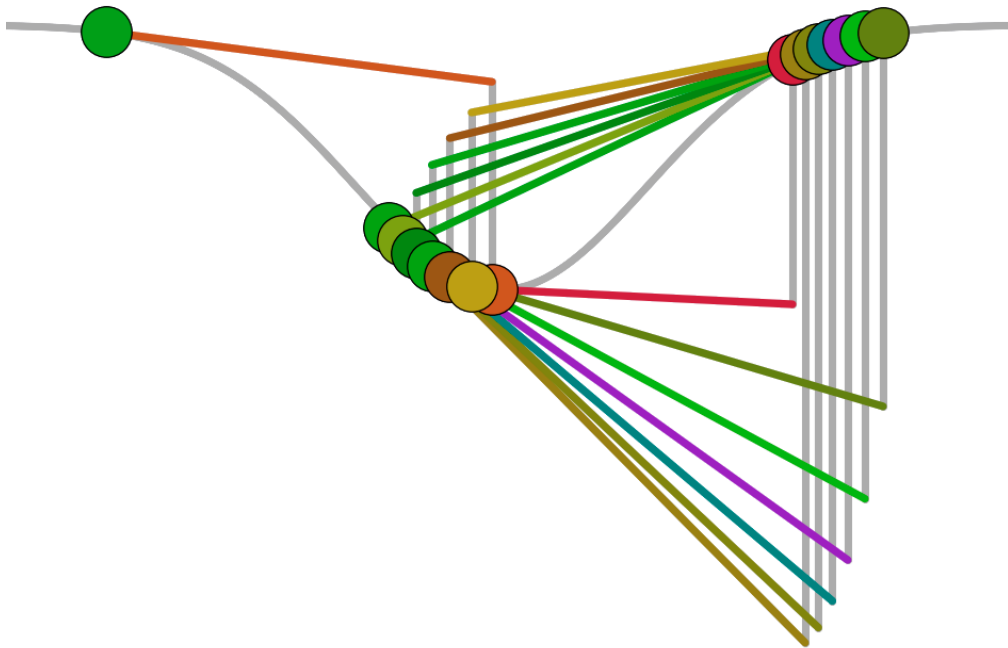


Learning rate $\epsilon = \frac{1}{2}$, 15 steps

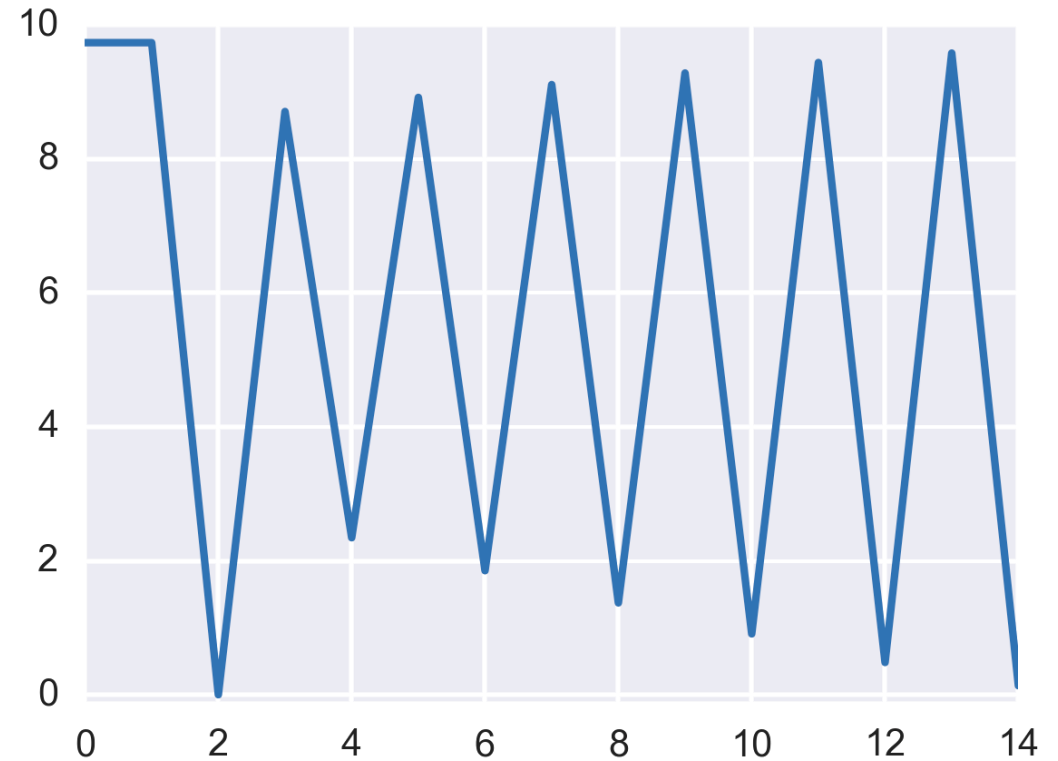


Error curve

Constant Learning Rate, Part IV

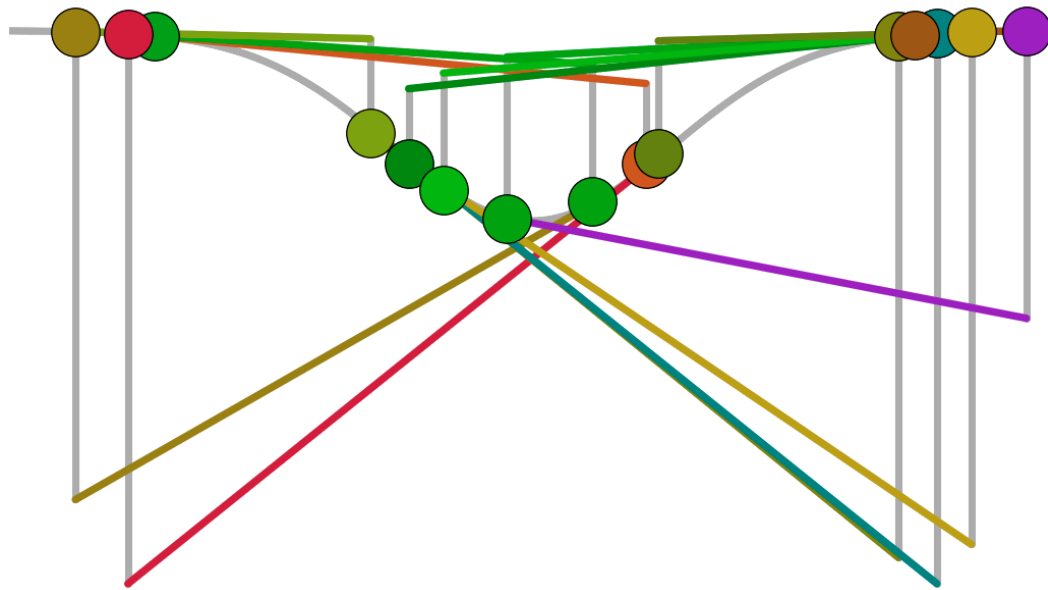


Learning rate $\epsilon = \frac{3}{4}$, 15 steps

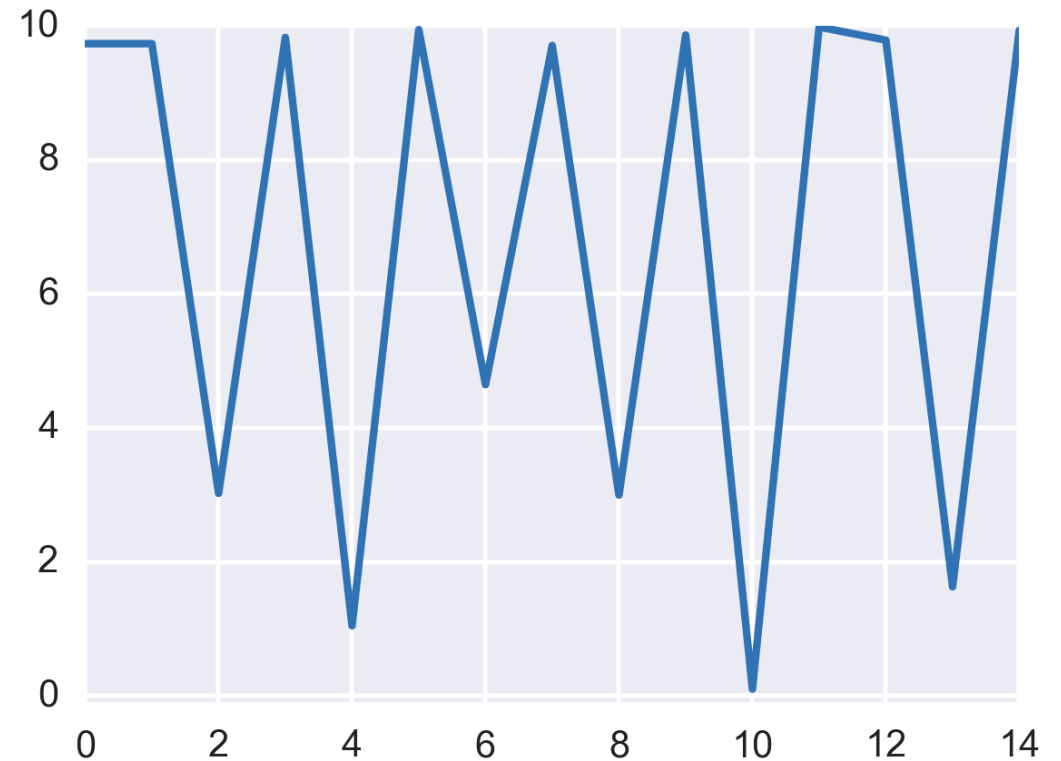


Error curve

Constant Learning Rate, Part V



Learning rate $\epsilon = 1.$, 15 steps



Error curve

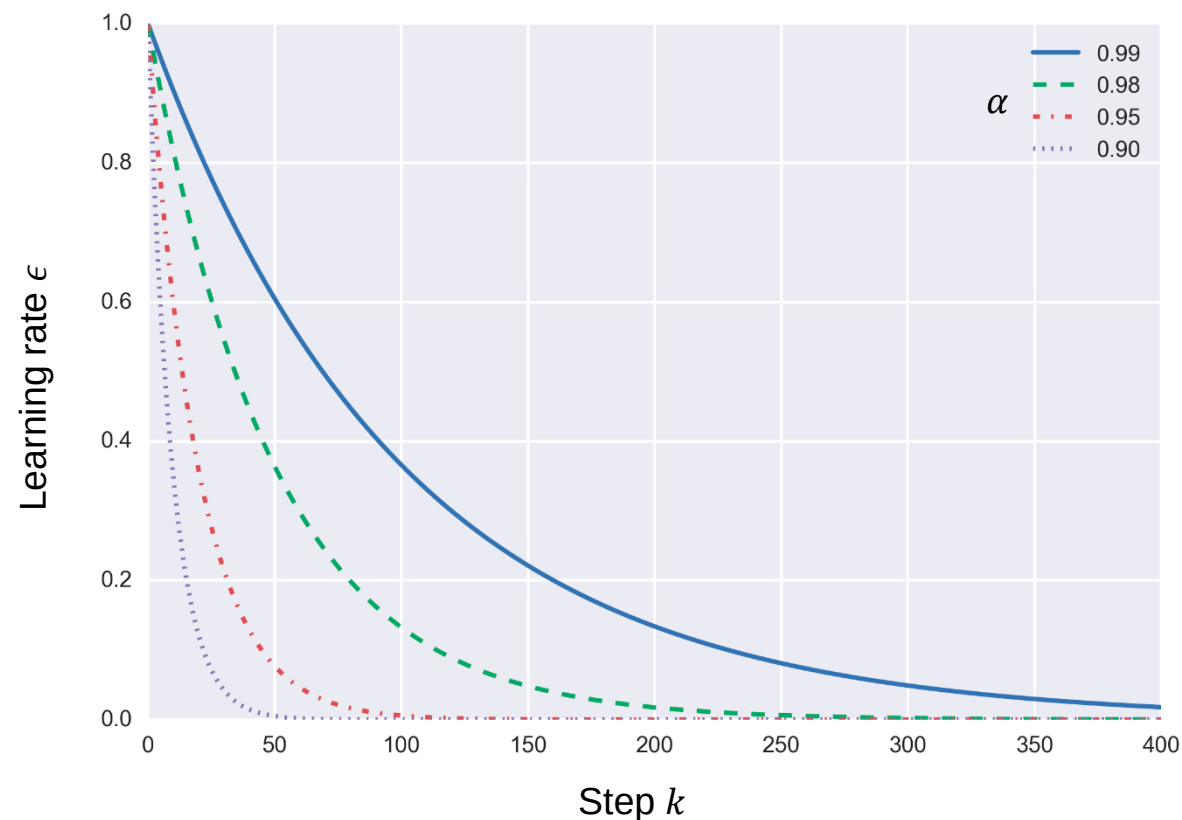
Adjusting Learning Rate

- A learning rate that is too high or too low will lead to **poor model performance**.
- In practice, learning rate is often tuned using a grid search or other **hyperparameter optimization** techniques.
- Some optimization algorithms **adapt the learning rate** during training to further improve performance.

Exponential Decay

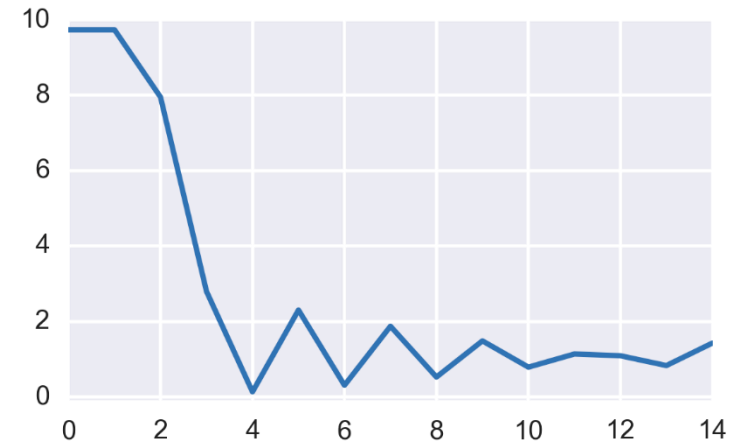
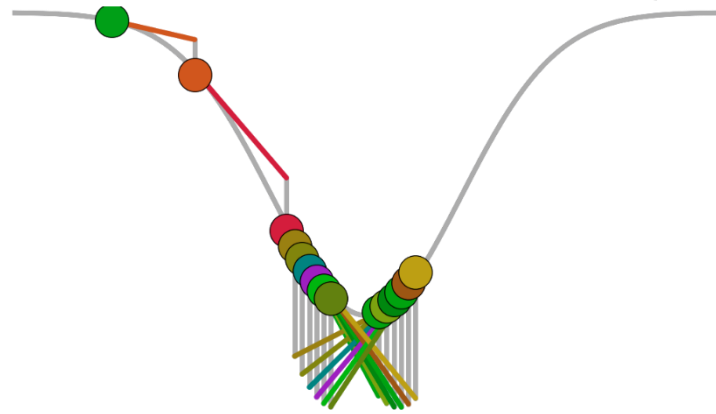
- Start with a high learning rate (the model can make quick progress).
- Gradually **decrease the learning rate** as the training progresses (exponentially).

$$\epsilon_k = \epsilon_0 \cdot \alpha^k$$

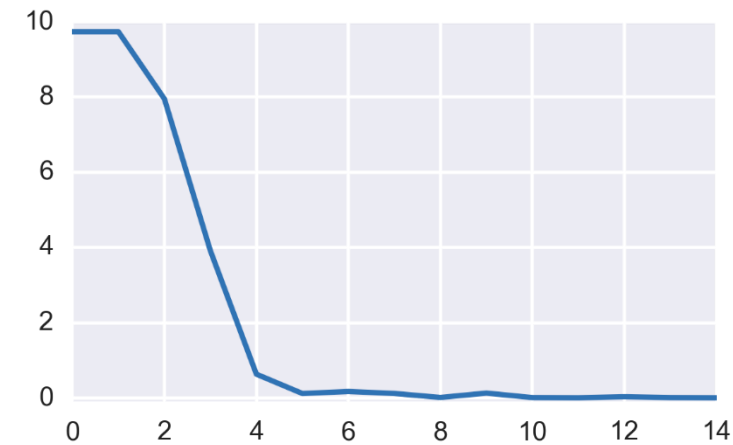
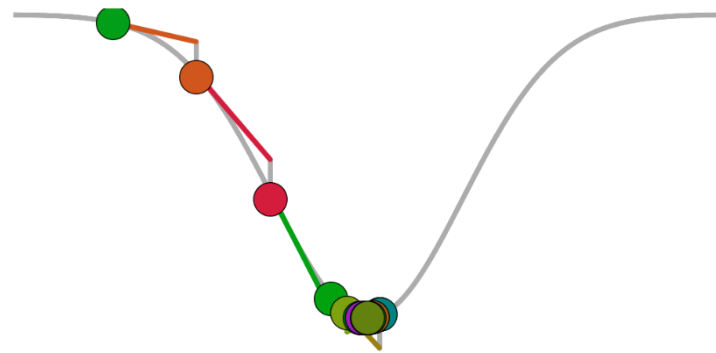


Exponential Decay vs. Constant Learning Rate

Constant
learning rate

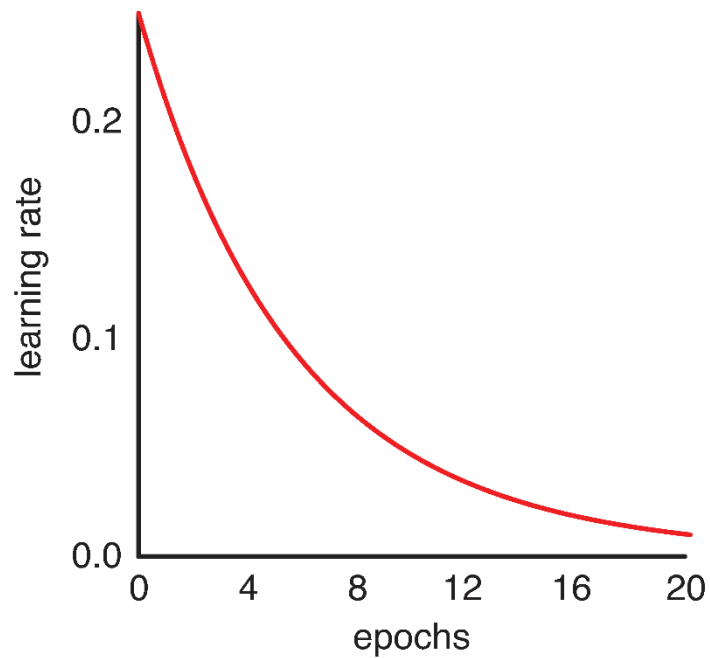


Exponential
decay

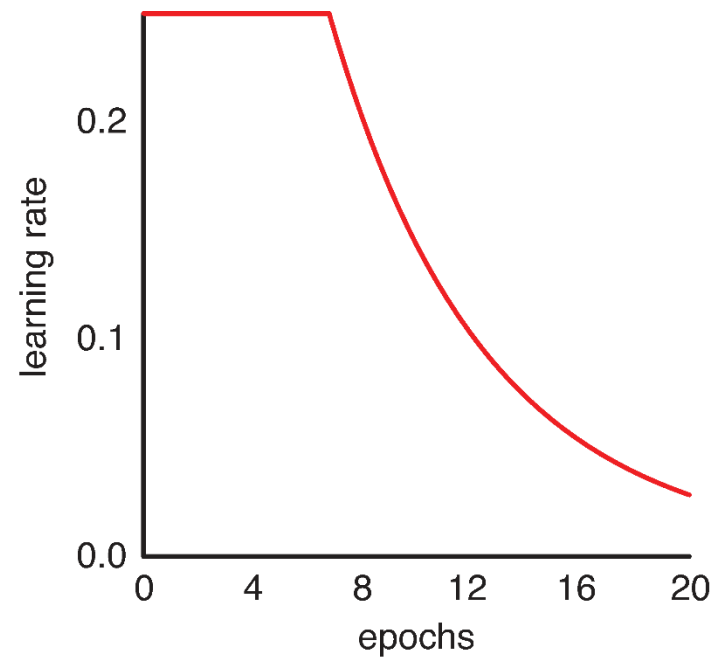


Error curves

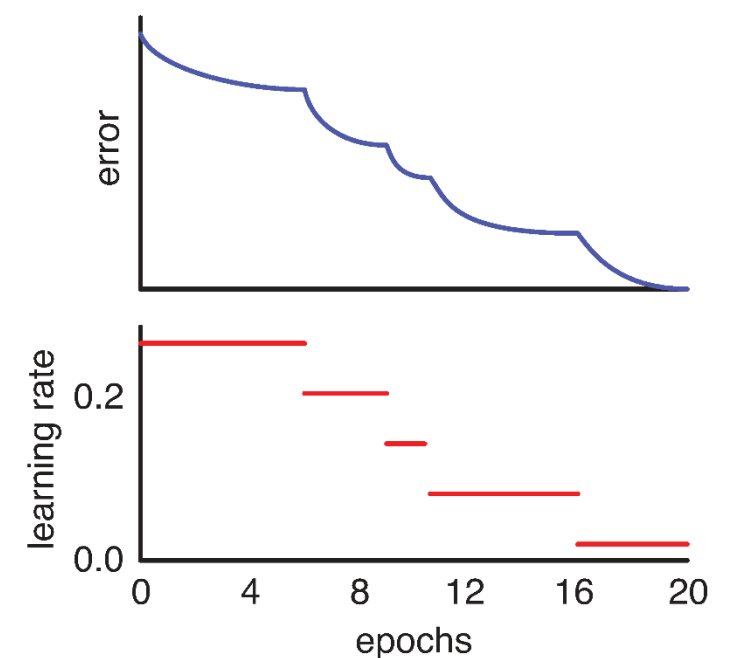
More Decay Schedules



Exponential decay



Delayed exponential
decay



Error-based decay

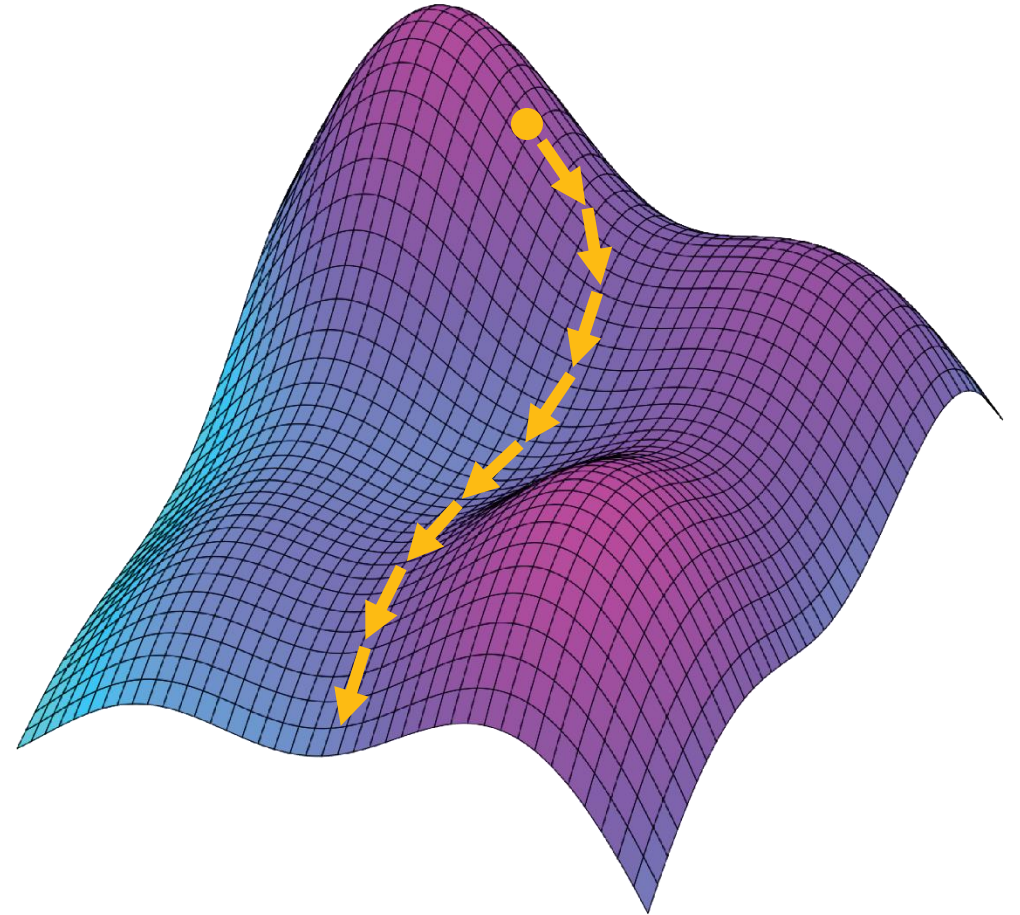
Learning Rate and Schedules

The End

Gradient Descent

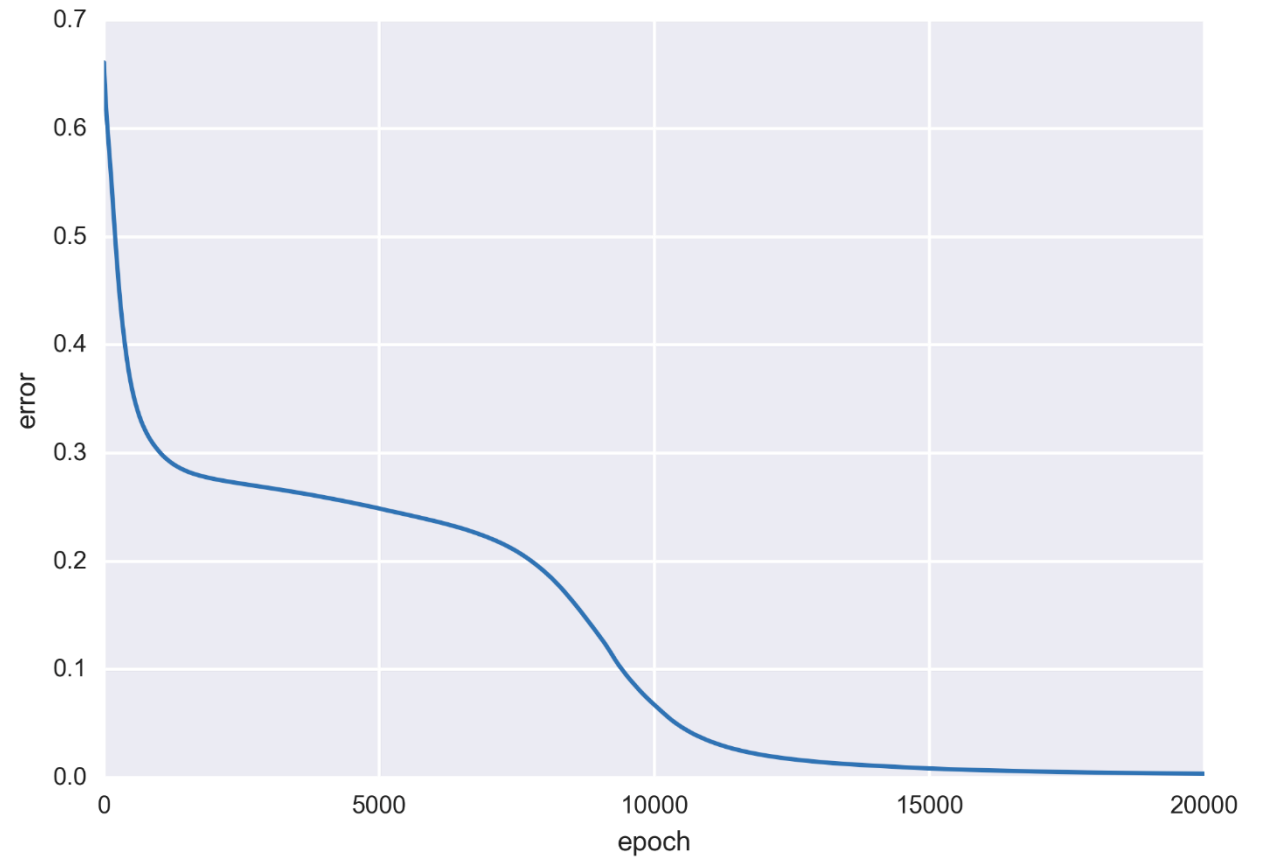
Weight-Updating Strategies

- Gradient descent variations
 - Batch gradient descent
 - Stochastic gradient descent
 - Mini-batch gradient descent
- **Primary difference:** the amount of data used to compute the gradients



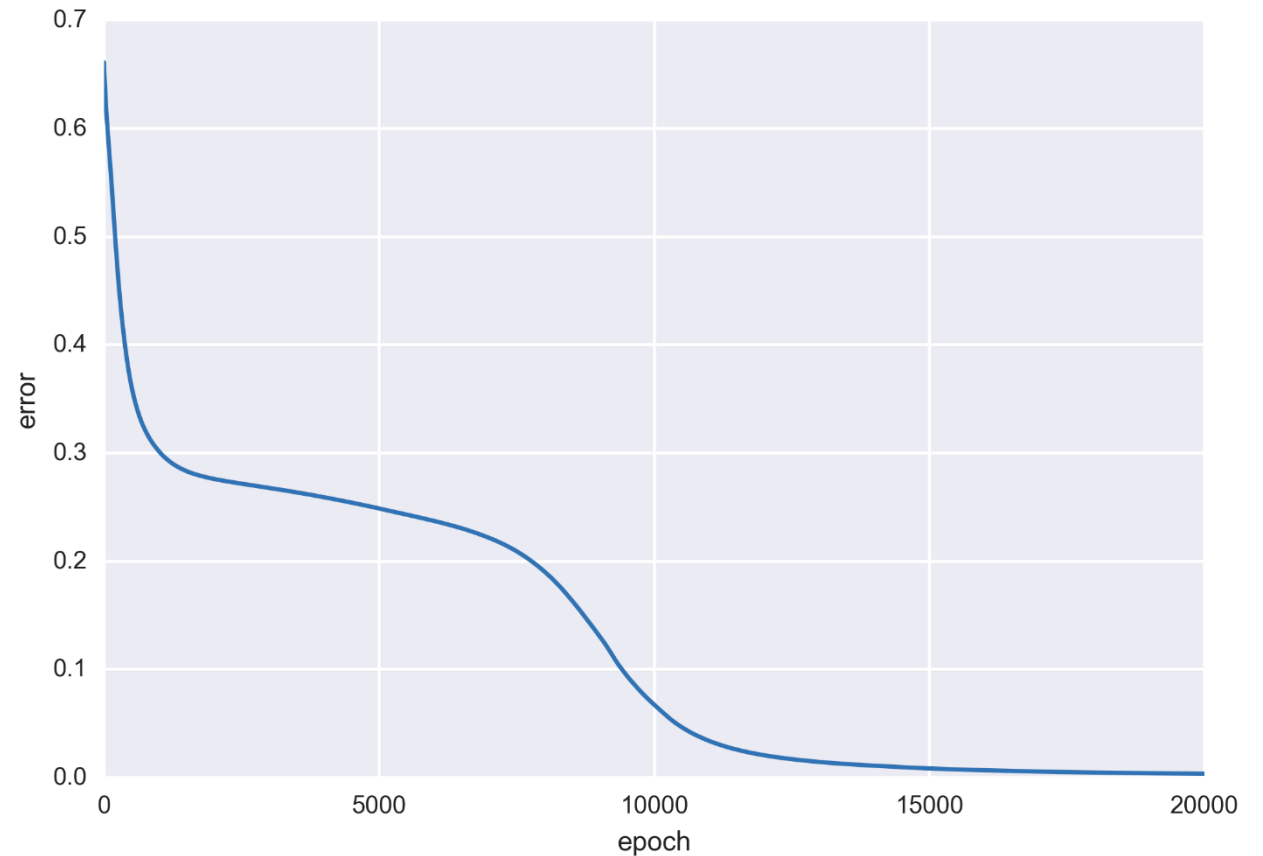
Batch Gradient Descent (BGD)

- The entire dataset is used to calculate the gradient of the cost function
- It **updates** the model parameters **once per epoch** (i.e., using average)



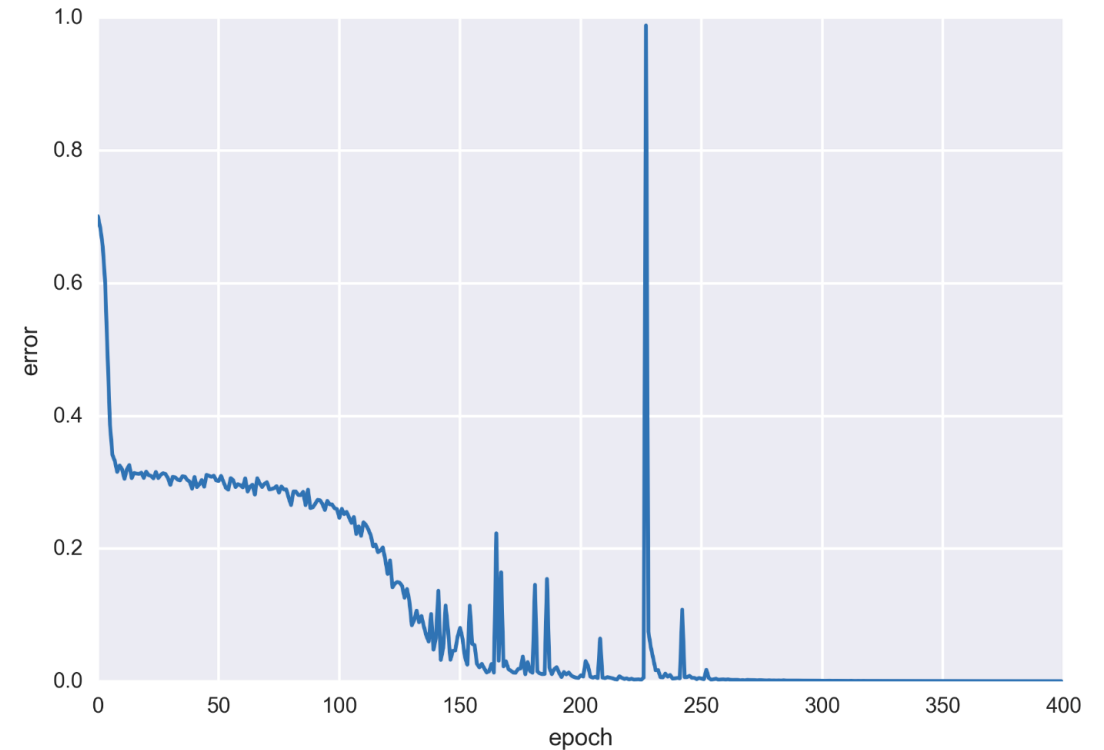
Batch Gradient Descent (BGD) (cont.)

- Smooth error curves
- Less sensitive to outliers
- Computationally expensive
- Issues with computer's memory for large datasets



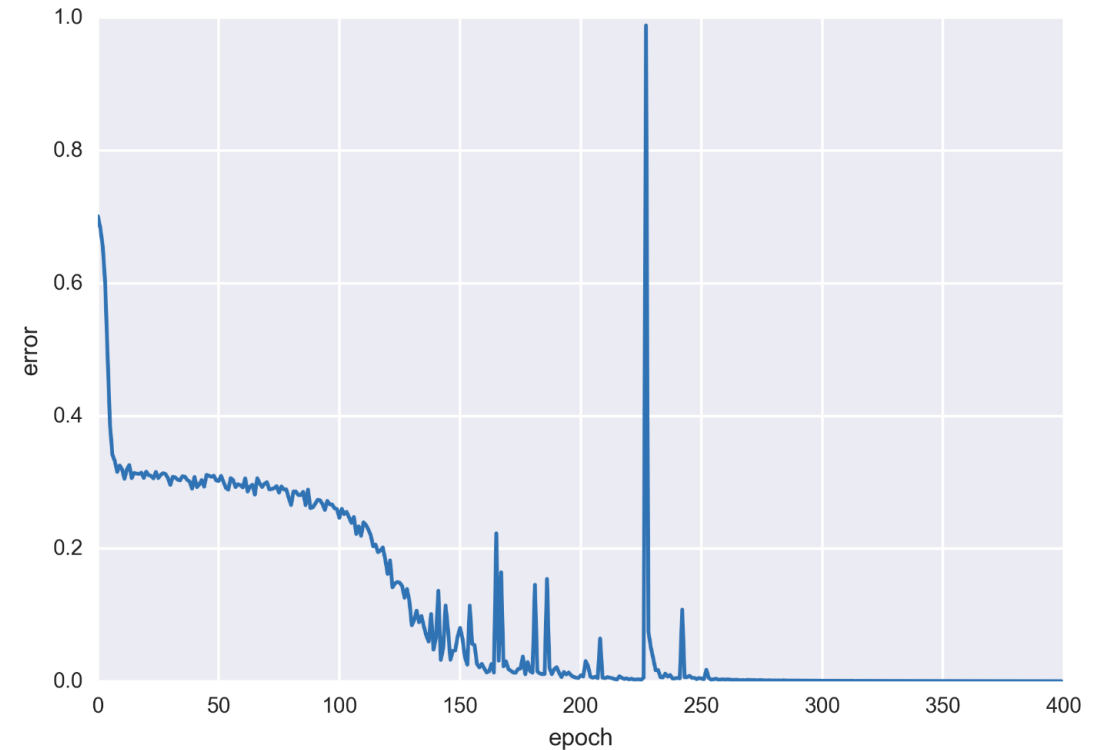
Stochastic Gradient Descent (SGD)

- Selects **only one random training sample**, computes the gradient, and updates the weights
- Process is repeated for every sample in the dataset until convergence



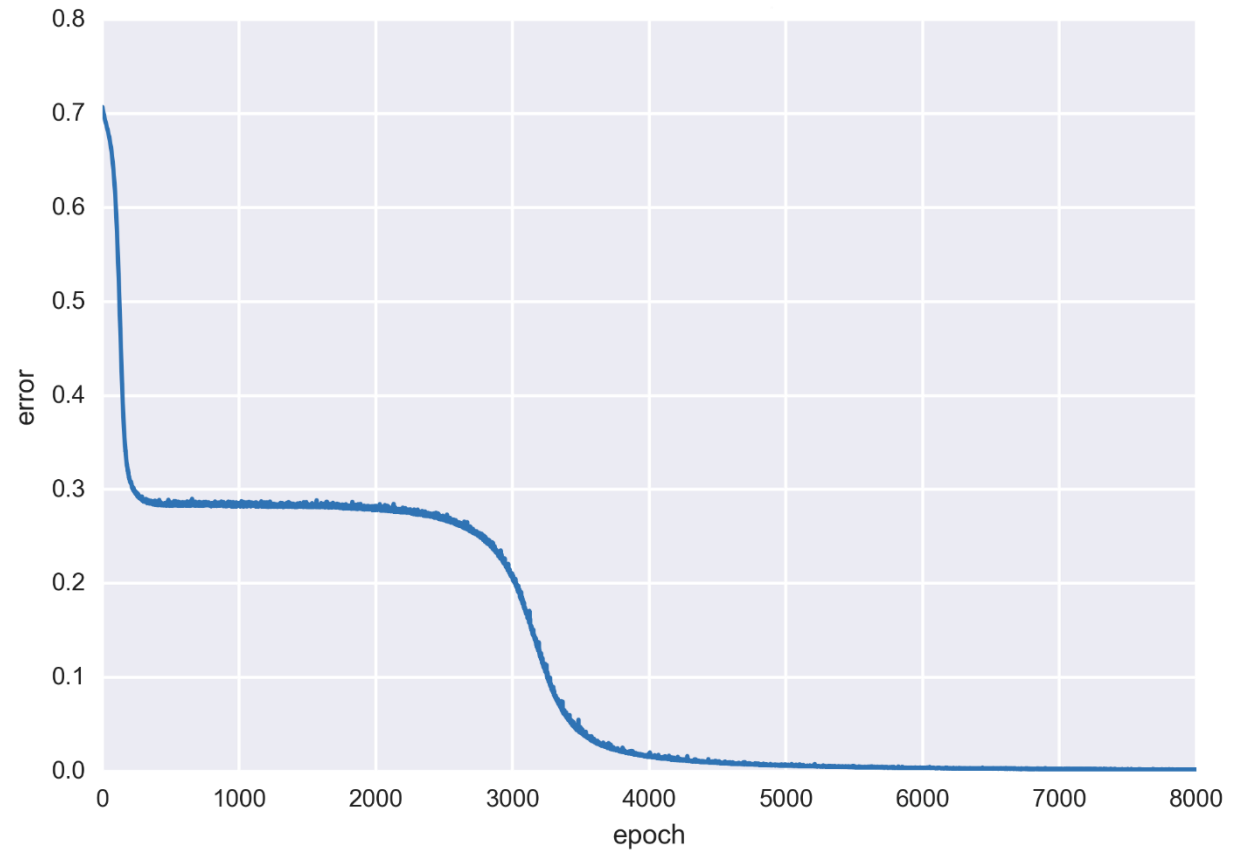
Stochastic Gradient Descent (SGD) (cont.)

- Computationally less expensive than BGD
- Random sample gradient updates makes error curve noisy
- Can be difficult to define a stopping criterion



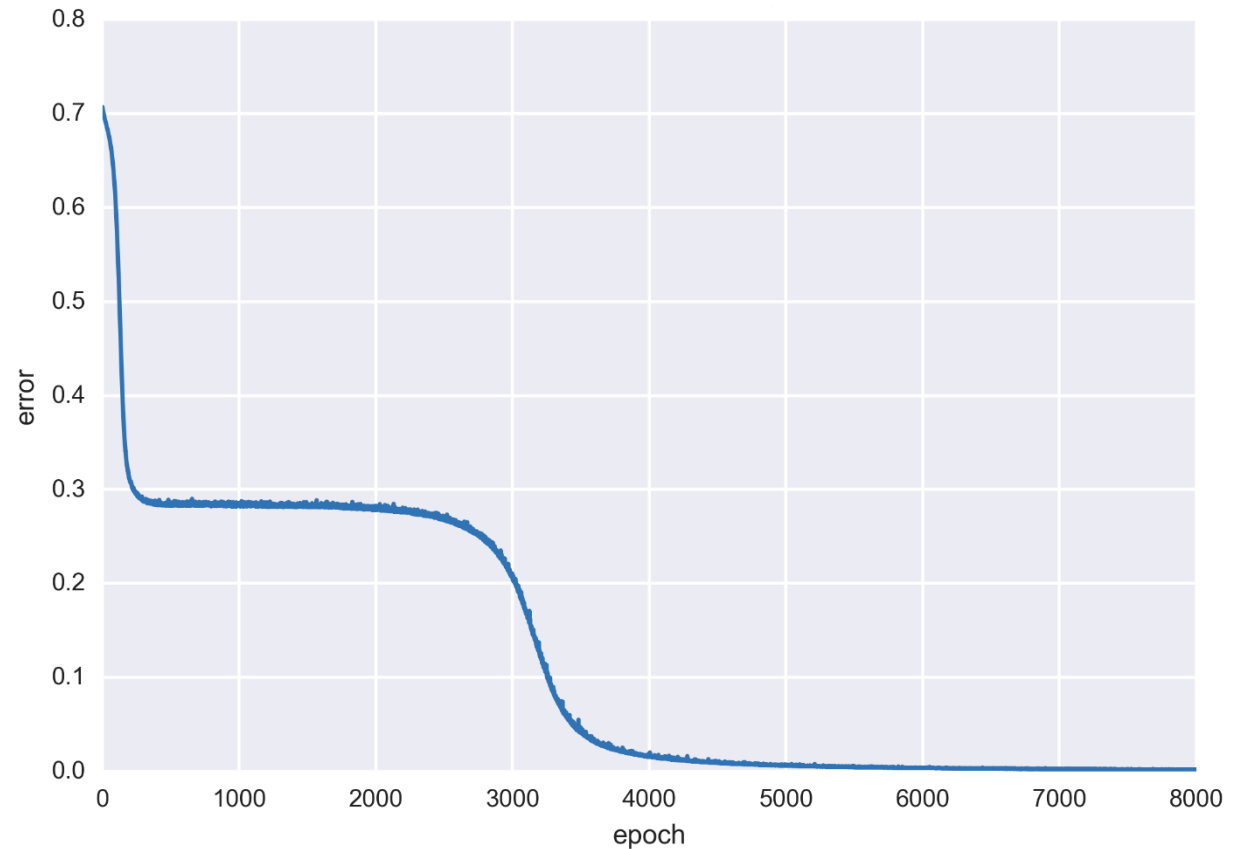
Mini-Batch Gradient Descent

- A **compromise** between batch gradient descent and stochastic gradient descent
- Training data are divided into **small batches**, computes the gradient, and updates the weights for each batch



Mini-Batch Gradient Descent (cont.)

- Less noisy than stochastic gradient descent
- Graphics processing units (GPUs) can process a mini-batch in parallel
- Most popular in practice

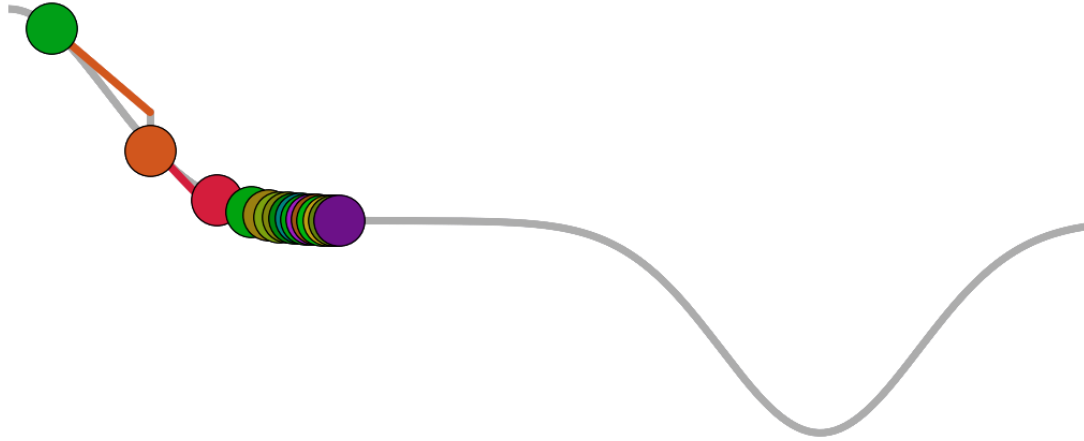


Gradient Descent

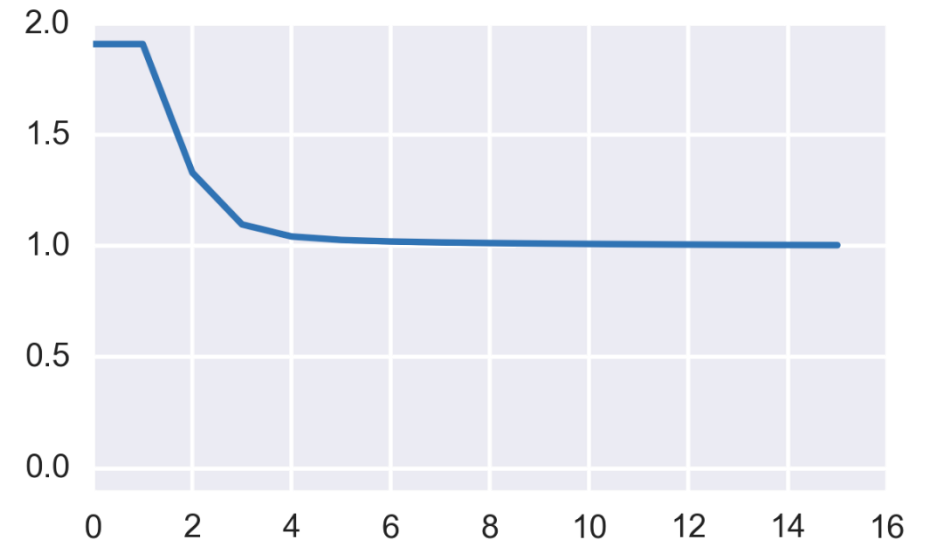
The End

Momentum

Gradient Descent (GD) Convergence

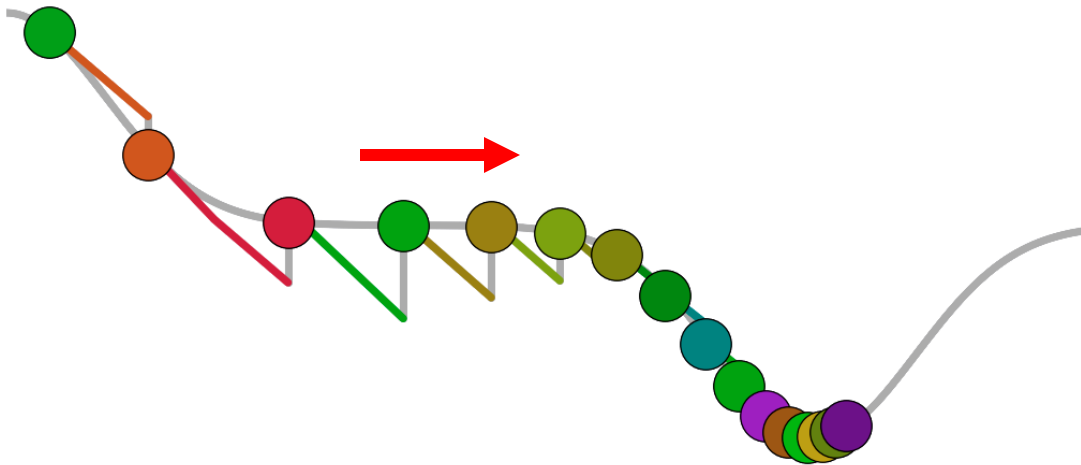


Gradient descent + Exponential decay

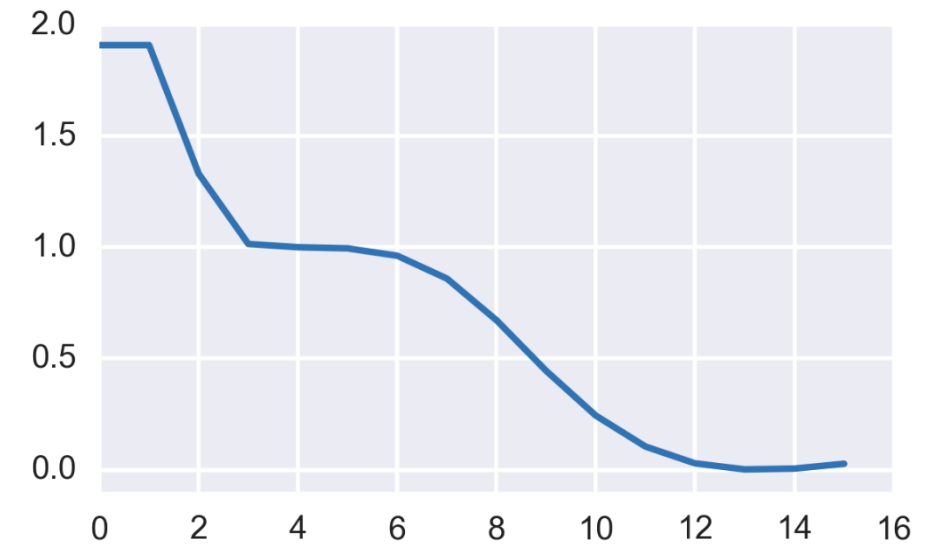


Error curve

Gradient Descent (GD) Convergence (cont.)



Momentum



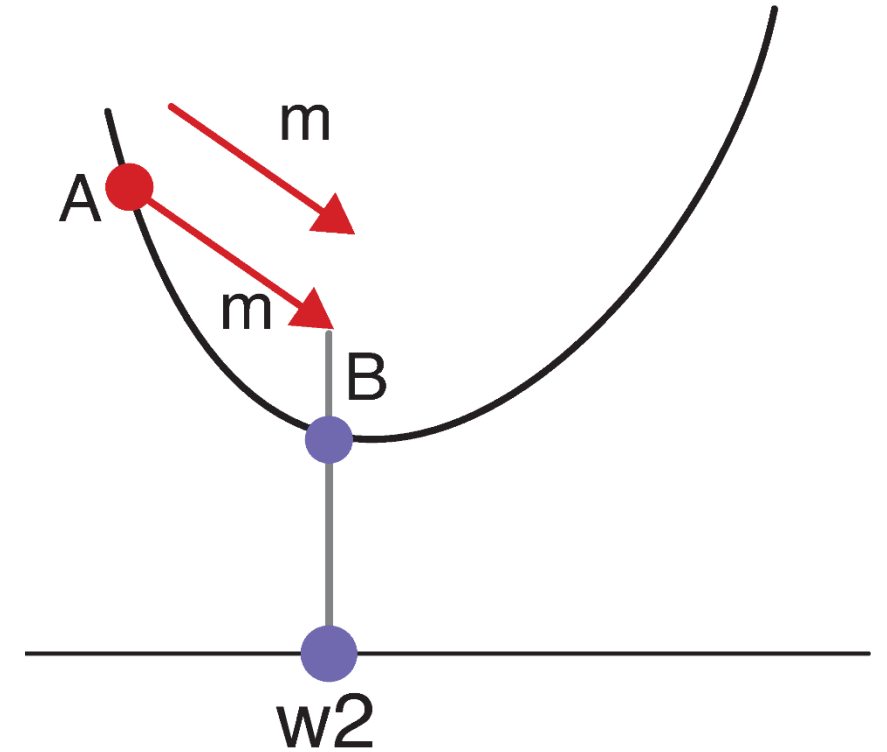
Error curve

Accelerating Convergence

- **Momentum** is a technique that can be used to accelerate the convergence of gradient descent algorithms.
- **Idea:** To add a fraction of the previous gradient, update to the current gradient update (**inertia**).
- It can help to **smooth** out the updates and get over flat plateaus and saddles.

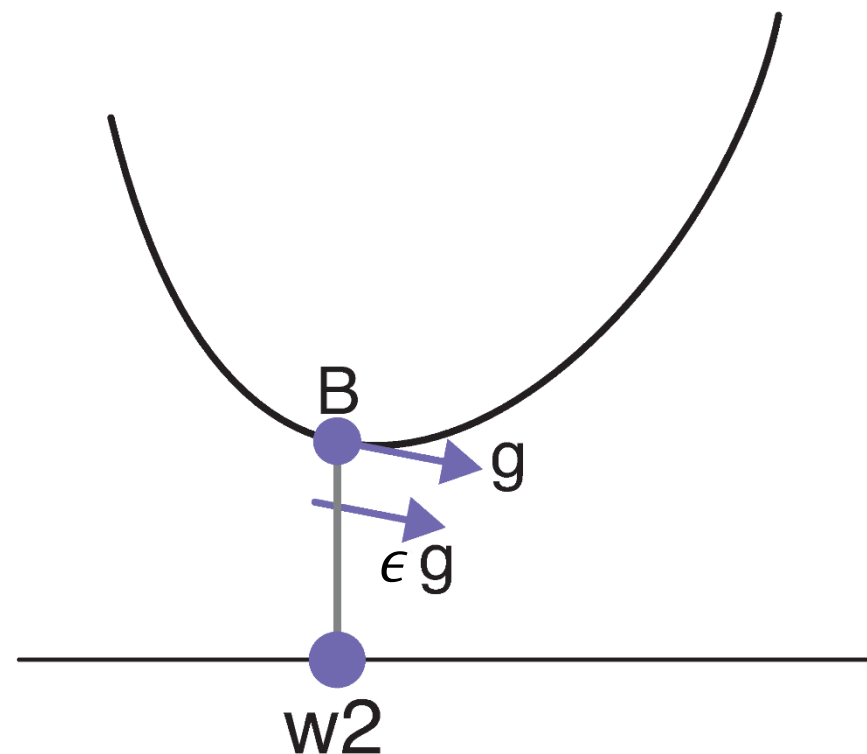
Momentum, Part I

- At point B, we look back and find the change we applied to point A (**momentum**).



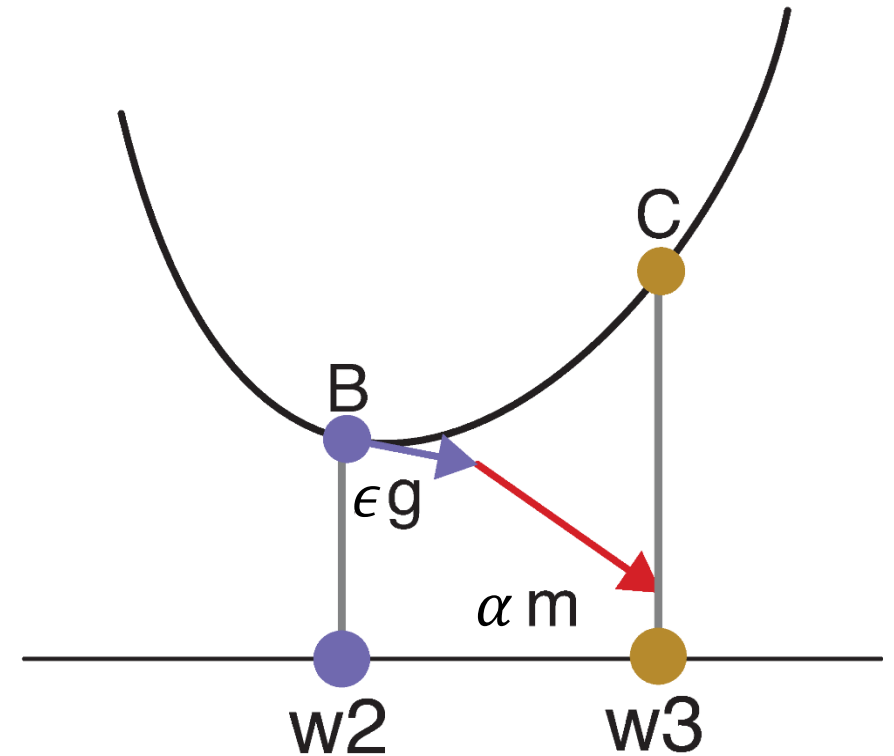
Momentum, Part II

- At point B, we look back and find the change we applied to point A (**momentum**).
- Using backpropagation, we find the gradient at B (and scale it by ϵ).



Momentum, Part III

- At point B, we look back and find the change we applied to point A (**momentum**).
- Using backpropagation, we find the gradient at B (and scale it by ϵ).
- We add the scaled gradient to the **scaled momentum** (α).



Formal Definition

- Momentum introduces a velocity parameter $\boldsymbol{v}$
- The velocity accumulates negative gradients

$$\boldsymbol{v}_t = \alpha \boldsymbol{v}_{t-1} - \epsilon \nabla f(\boldsymbol{x}_t)$$

- Gradient descent with momentum updates

$$\boldsymbol{x}_{t+1} = \boldsymbol{x}_t + \boldsymbol{v}_t$$

Momentum

The End

Algorithms With Adaptive Learning Rates

Adaptive Learning Rates

- Learning rate is one of the hyperparameters that is most difficult to set
- Significant impact on the model performance
- Momentum mitigates its impact but introduces another hyperparameter!
- Idea: **adapting learning** rates for each weight during training

AdaGrad, Part I

- Adaptive gradient learning algorithm **individually changes the size of the gradient** for each weight
- **Scale factor:** inversely proportional to the square root of the sum off all their historical squared values

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \epsilon \nabla f(\mathbf{x}_t)$$

GD update rule

AdaGrad, Part II

- Adaptive gradient learning algorithm **individually changes the size of the gradient** for each weight
- **Scale factor:** inversely proportional to the square root of the sum off all their historical squared values

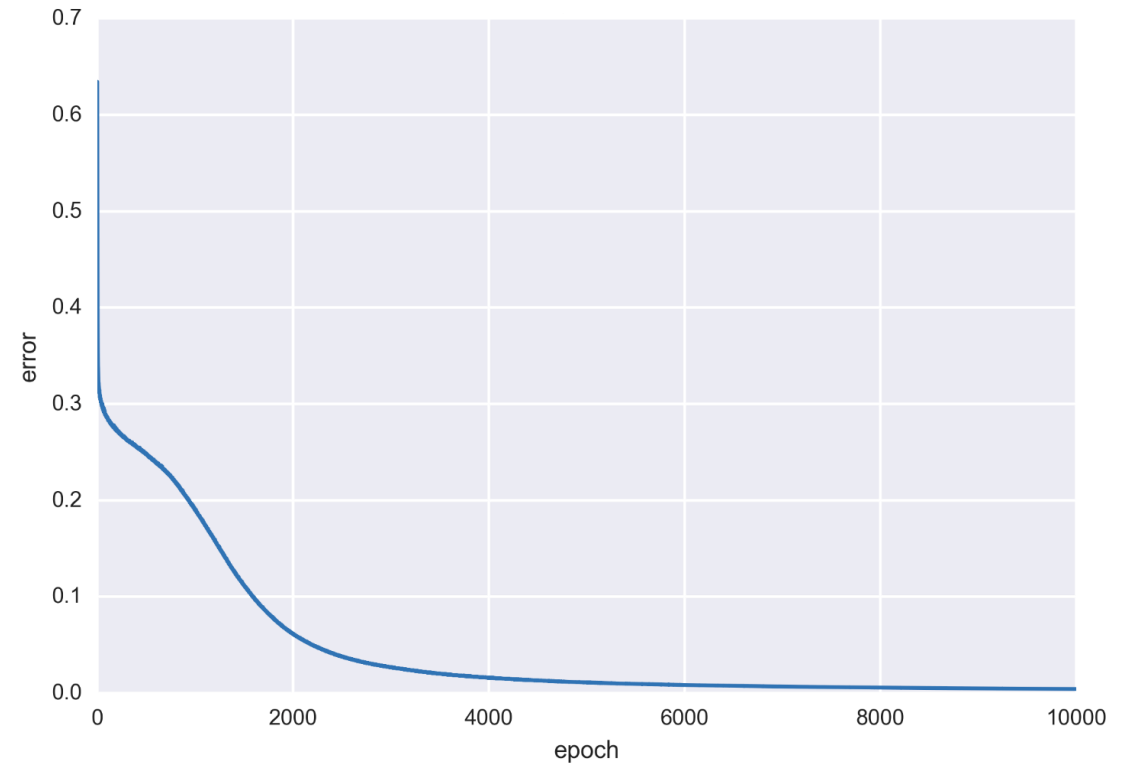
$$\mathbf{r}_t = \mathbf{r}_{t-1} + \nabla f(\mathbf{x}_t) \odot \nabla f(\mathbf{x}_t)$$

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \frac{\epsilon}{\delta + \sqrt{\mathbf{r}_t}} \odot \nabla f(\mathbf{x}_t)$$

AdaGrad update rule

AdaGrad, Part III

- More progress in directions with low curvature
- Allows for progress in higher curvature regions
- **Problem:** accumulated sum r gets larger over time and gradients become insignificant (approach 0)



Adadelta and RMSprop

- Idea: Use a **decaying sum** of gradients instead of accumulating all history.
- Recent gradient values have more influence than old ones.
- It tends to converge faster than AdaGrad.
- **Downside:** We introduced another parameter (γ) defining how much we scale down the gradient in the history list.

Adaptive Moment Estimation (ADAM), Part I

- Squaring the gradient in AdaGrad/Adadelata/RMSprop is usefull mathematically but always positive; we lose track of the gradient sign
- ADAM keeps a second sum of gradients without squaring them; the scaling factor is derived using both lists

$$\mathbf{s}_t = \beta_1 \mathbf{s}_{t-1} + (1 - \beta_1) \nabla f(\mathbf{x}_t)$$

$$\mathbf{r}_t = \beta_2 \mathbf{r}_{t-1} + (1 - \beta_2) \nabla f(\mathbf{x}_t) \odot \nabla f(\mathbf{x}_t)$$

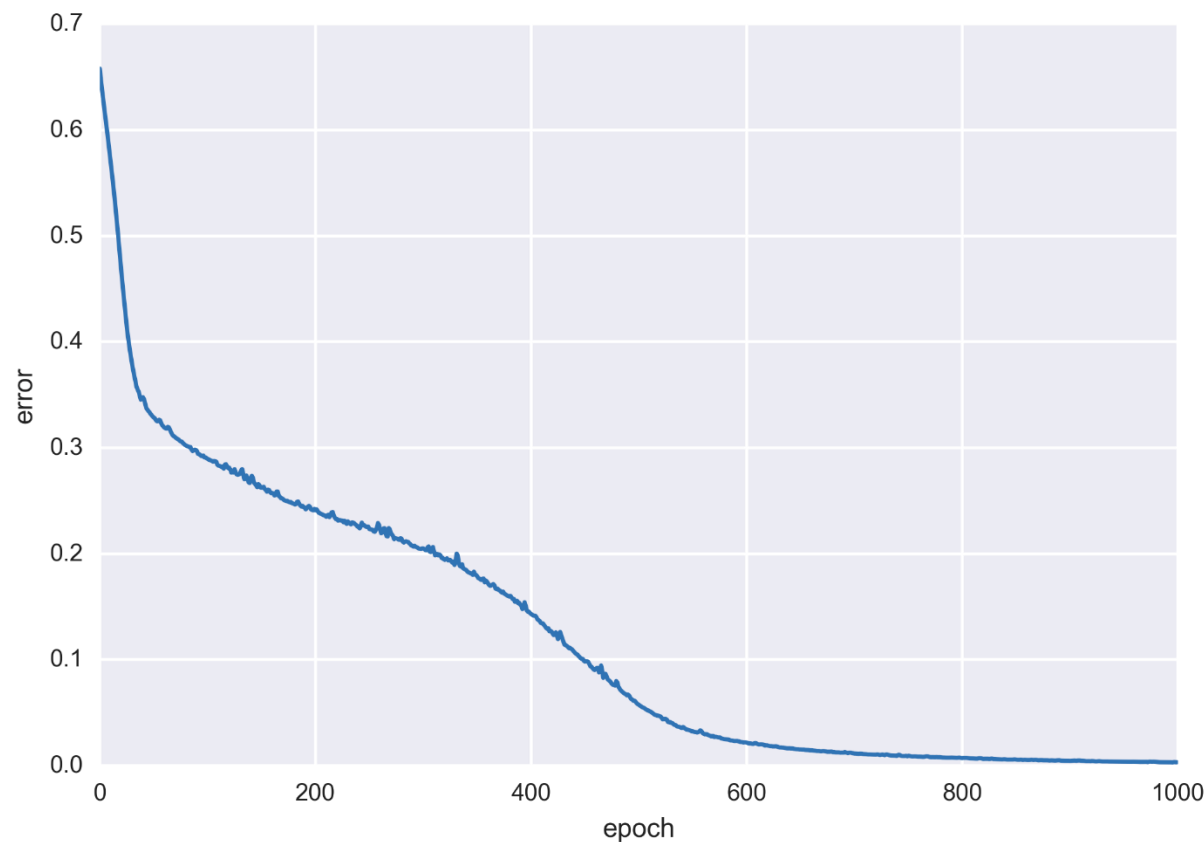
Adaptive Moment Estimation (ADAM), Part II

- Squaring the gradient in AdaGrad/Adadelta/RMSprop is useful mathematically but always positive; we lose track of the gradient sign
- ADAM keeps a second sum of gradients without squaring them; the scaling factor is derived using both lists

$$\mathbf{x}_{t+1} = \mathbf{x}_t - \epsilon \frac{\mathbf{s}_t}{\delta + \sqrt{\mathbf{r}_t}}$$

Adaptive Moment Estimation (ADAM), Part III

- Faster than AdaGrad or Adadelta; widely used in deep neural network training
- Downside: two extra parameters, β_1 and β_2 (decay for each sum)



Algorithms With Adaptive Learning Rates

The End

Choosing the Right Optimizer

No Free Lunch

- Why are there so many optimization algorithms? Why not find the best one and stick to it?
- There is no “best” optimizer: There can’t be a best optimizer for all situations (**no free lunch theorem, 1997**).
- We can find the best optimizer for a specific network and data combination, trying multiple algorithms and hyperparameters.

Choosing the Right Optimizer

The End